Lecture 2. Graph Algorithms

FALL 2026, *INTRODUCTION TO COMPUTER SCIENCE*, IIIS, TSINGHUA

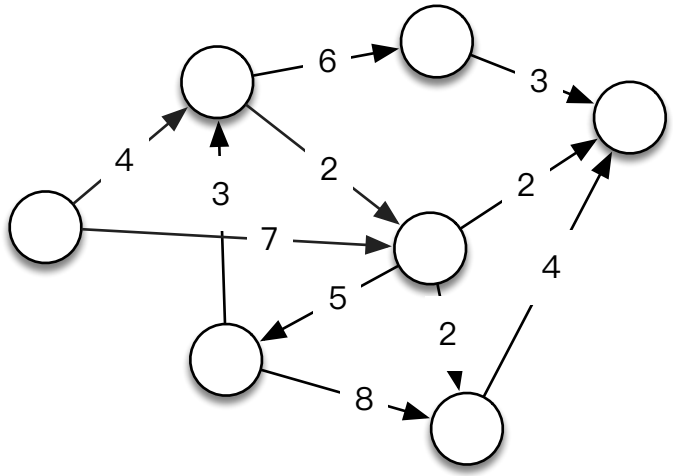
Graphs

Graph G=(V, E, w)

- V: set of vertices
- E: set of edges, that is, $E \subseteq \{(u,v) \mid u,v \in V\}$
- w: $E \to \mathbb{R}$
- Can denote the length of the edge

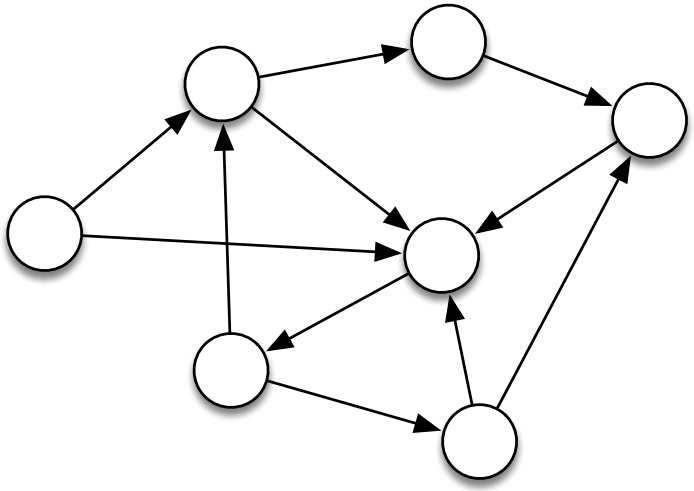
Degree of u: number of edges associated with u

- Denoted by deg(u)

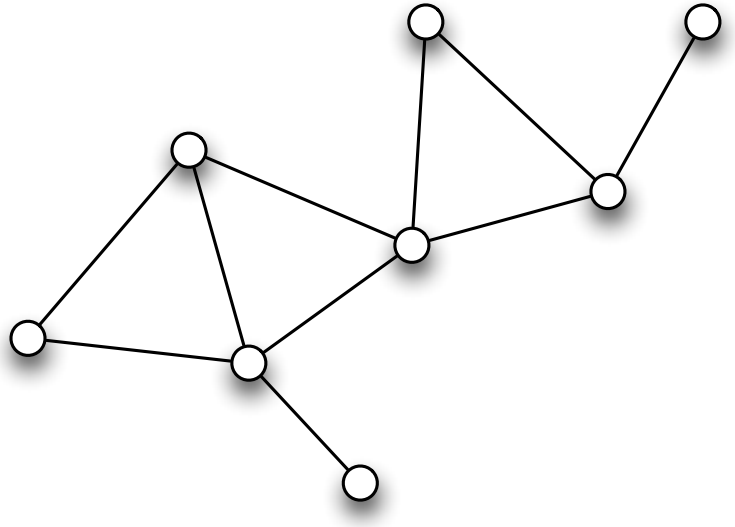


Graphs

DIRECTED GRAPHS

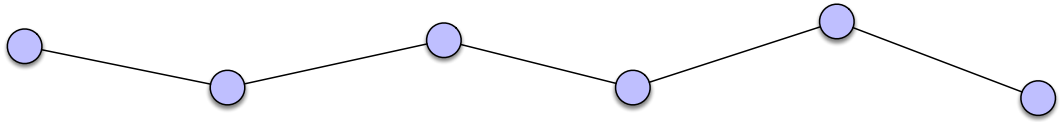


UNDIRECTED GRAPHS

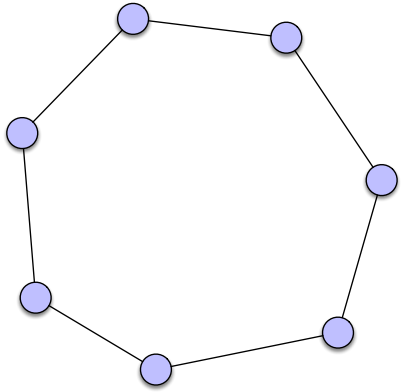


Path and cycle

Path: $v_0 \rightarrow v_1 \rightarrow \ldots \rightarrow v_k$



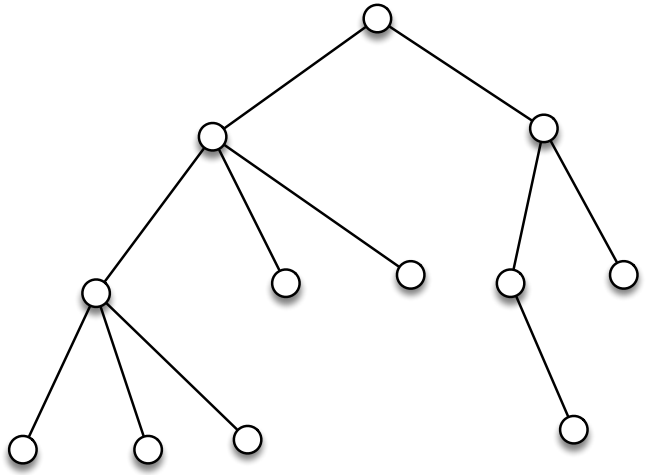
Cycle $v_0 = v_k$



Simple path/simple cycle: no repetition of vertices

Tree

A *tree* is an undirected connected and acyclic (contains no cycles) graph *G (with n vertices)*

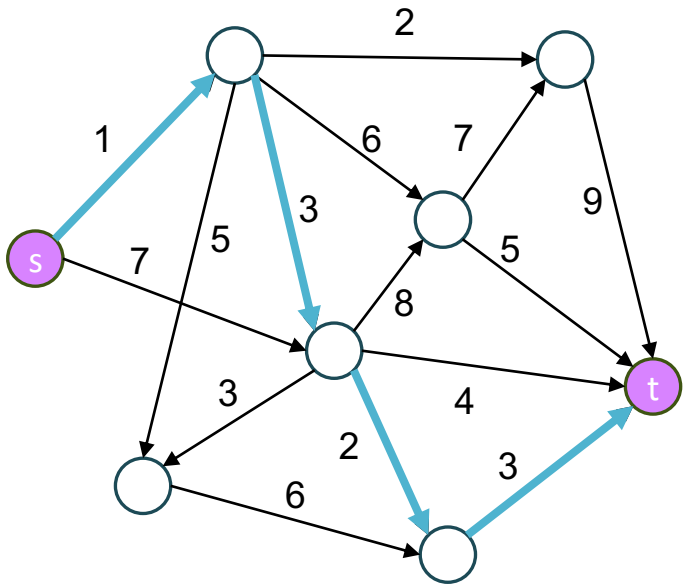


Shortest Path Problem

Directed/undirected graph G=(V, E, w)

- w: E→$\mathbb{R}_{\geq 0}$
- m=|E|, n=|V|

The length of the shortest path is called the distance, denoted by dist(s,t)

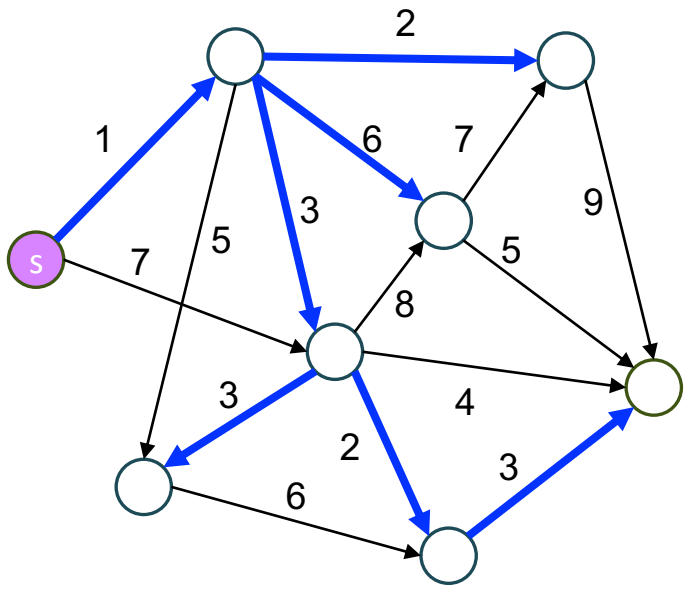


s-t shortest path

Single-Source Shortest Path

Single-source shortest path (SSSP): find shortest path from source s to all other vertices

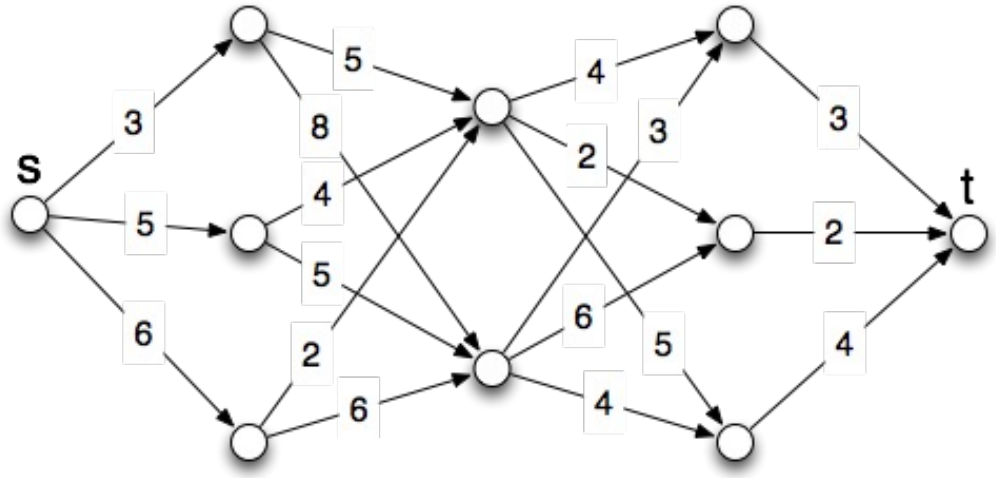
- it will form a *shortest path tree* from s



Shortest path tree

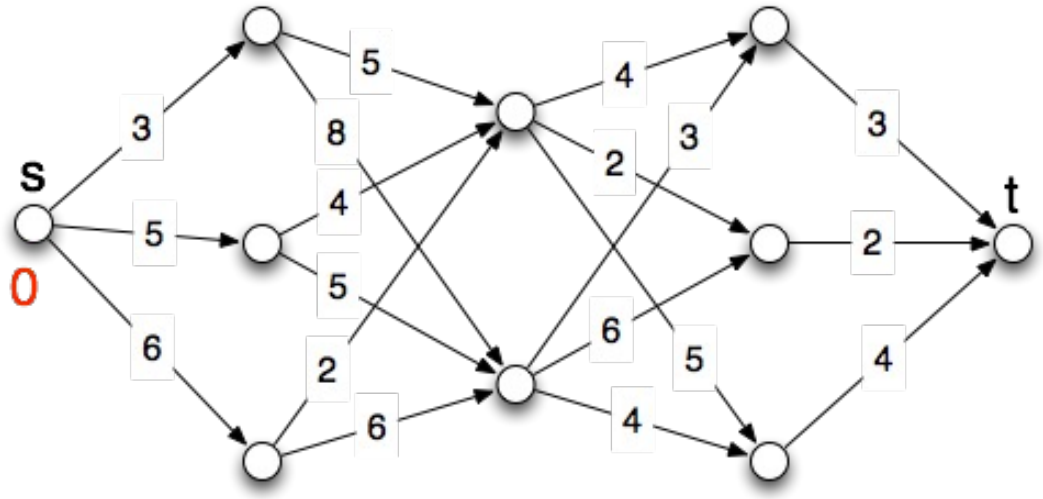
Shortest Path in Linear Time

Directed acyclic graph (DAG): dynamic programming



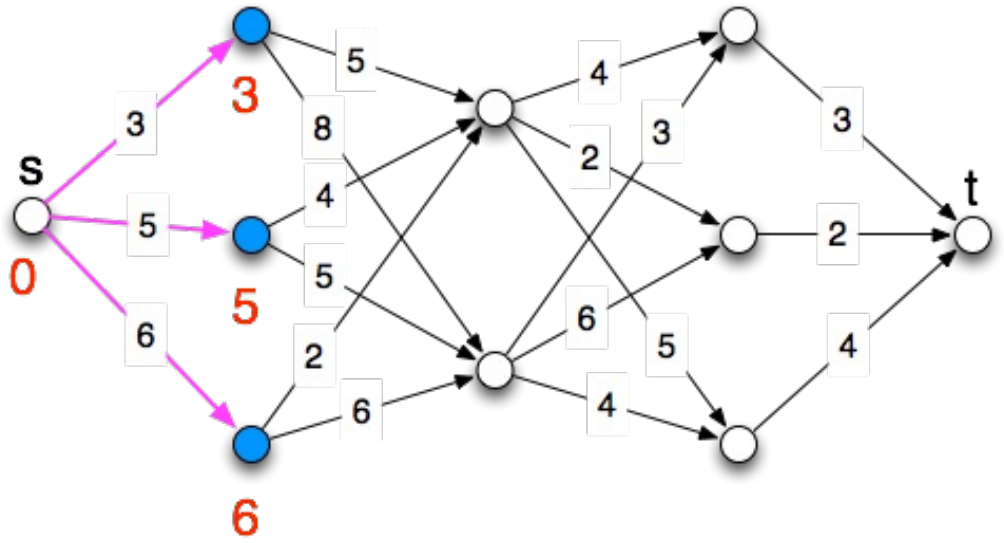
Shortest Path in Linear Time

Directed acyclic graph (DAG): dynamic programming



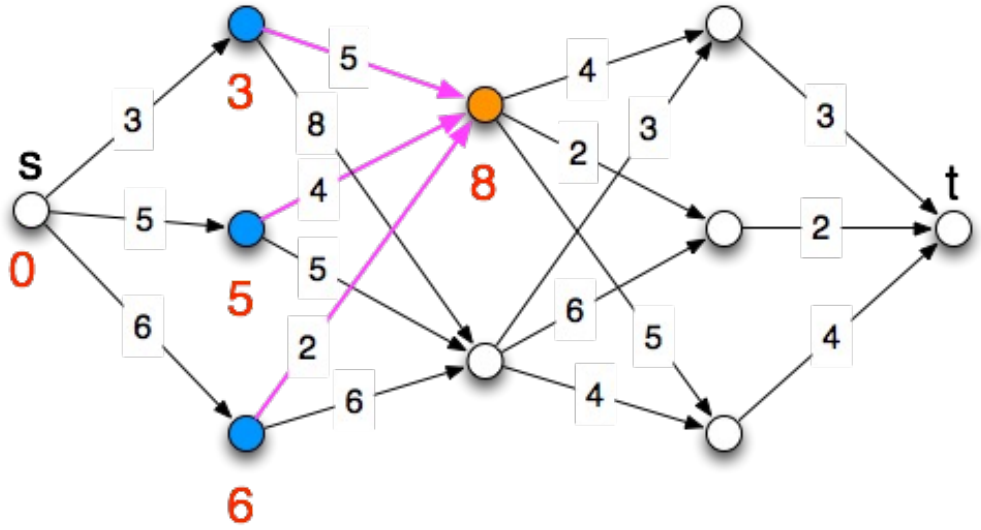
Shortest Path in Linear Time

Directed acyclic graph (DAG): dynamic programming



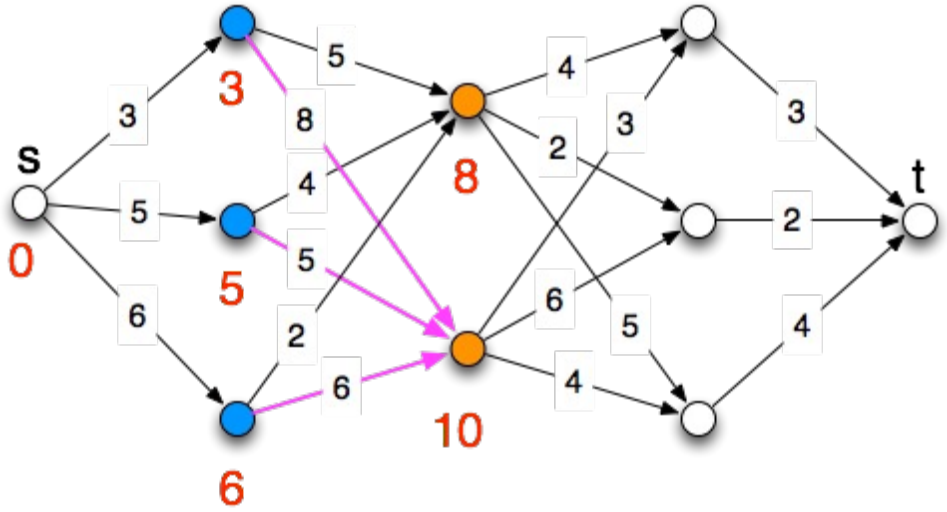
Shortest Path in Linear Time

Directed acyclic graph (DAG): dynamic programming



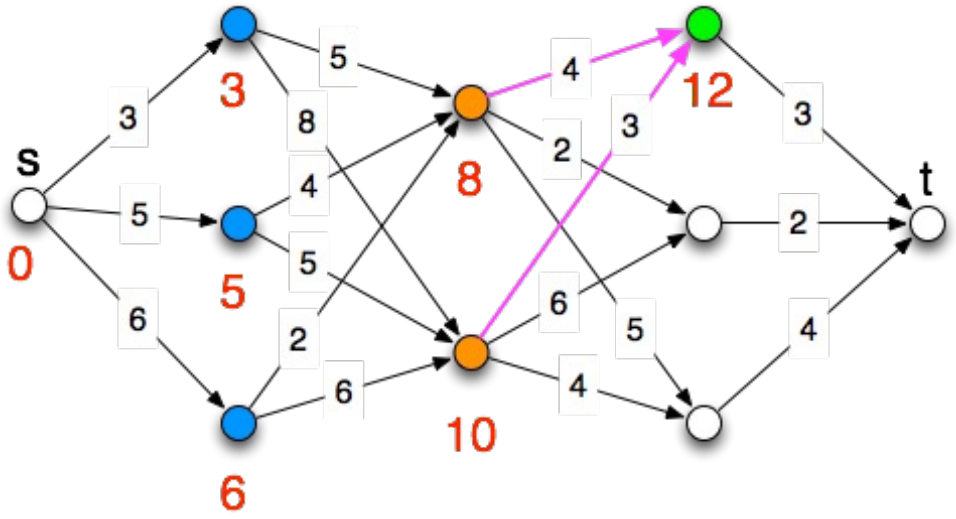
Shortest Path in Linear Time

Directed acyclic graph (DAG): dynamic programming



Shortest Path in Linear Time

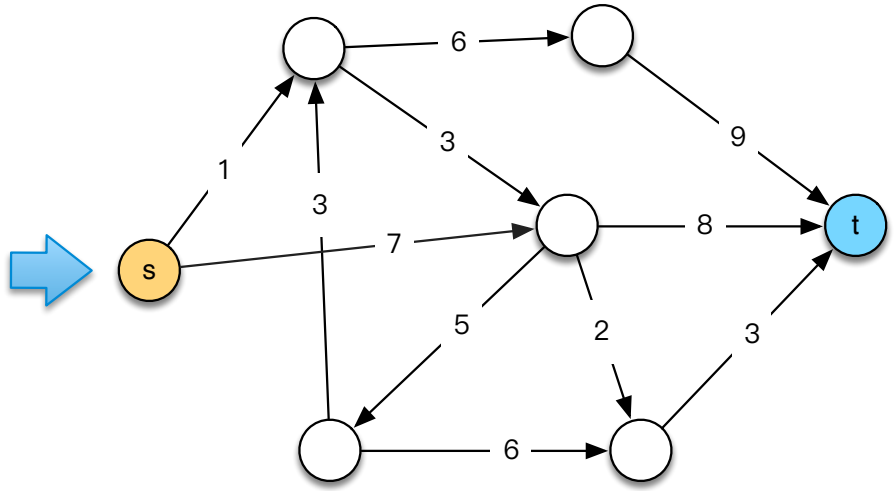
Directed acyclic graph (DAG): dynamic programming



Shortest Path in Linear Time

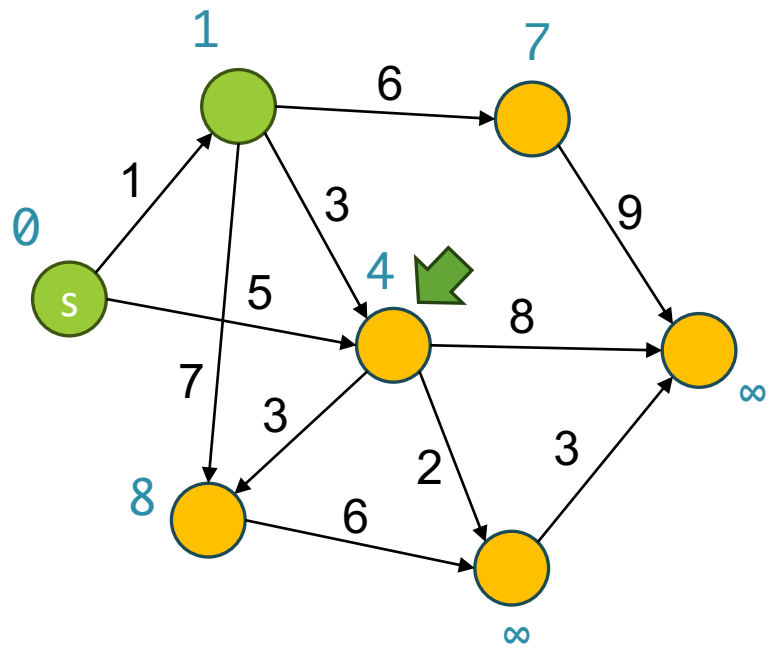
Directed acyclic graph (DAG): dynamic programming

General case: we don’t know the scanning order in advance



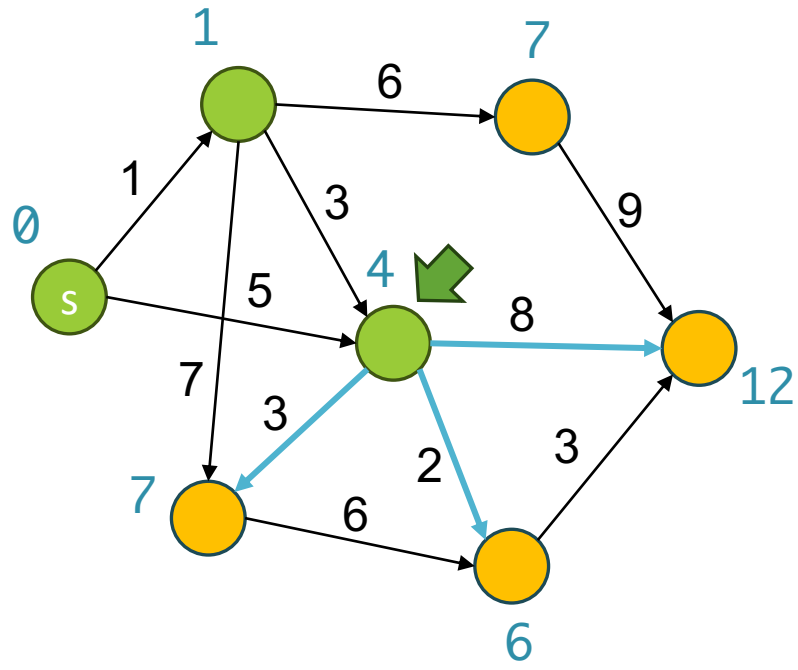
Single-source shortest path -- Dijkstra's algorithm

```
Label: d(s)←0, d(v)←+∞ for all other v
Mark all vertices active
While there is an active vertex
    Select an active u with minimum d(u)
    mark u complete
    Relax every outgoing edge (u,v):
        d(v)← min{d(v), d(u)+w(u,v)}
```



Single-source shortest path -- Dijkstra's algorithm

```
Label: d(s)←0, d(v)←+∞ for all other v
Mark all vertices active
While there is an active vertex
    Select an active u with minimum d(u)
    mark u complete
    Relax every outgoing edge (u,v):
        d(v)← min{d(v), d(u)+w(u,v)}
```



u is **complete**: d(u)=dist(s,u)

Correctness of Dijkstra

Key: when u is popped as the minimum active vertex,
d(u)=dist(s,u)

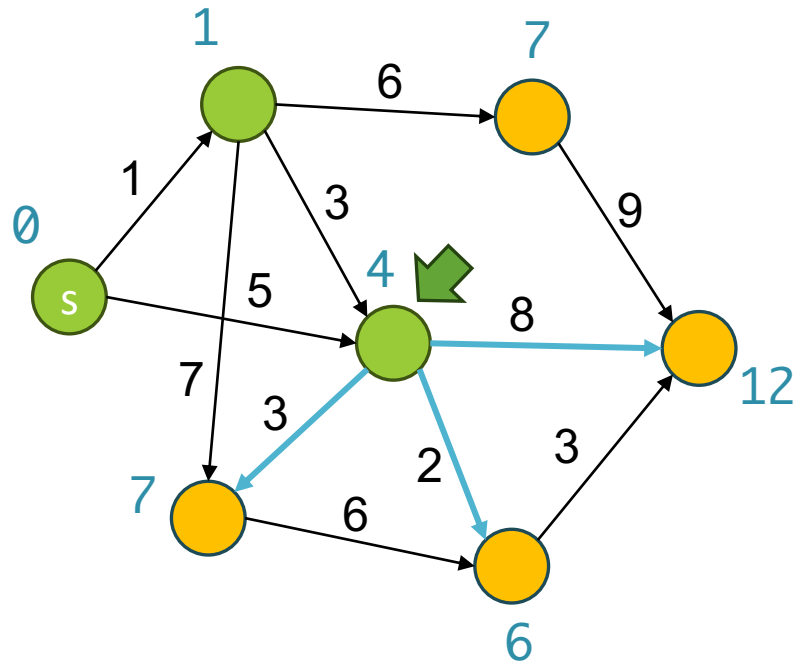
so we call them “complete”

You can prove by induction

```
Label: d(s)←0, d(v)←+∞ for all other v
Mark all vertices active
While there is an active vertex
    Select an active u with minimum d(u)
    mark u complete
    Relax every outgoing edge (u,v):
        d(v)← min{d(v), d(u)+w(u,v)}
```

Single-source shortest path -- Dijkstra's algorithm

```
Label: d(s)←0, d(v)←+∞ for all other v
Mark all vertices active
While there is an active vertex
    Select an active u with minimum d(u)
    mark u complete
    Relax every outgoing edge (u,v):
        d(v)← min{d(v), d(u)+w(u,v)}
```



Every time we can select the minimum active vertex in O(n) time

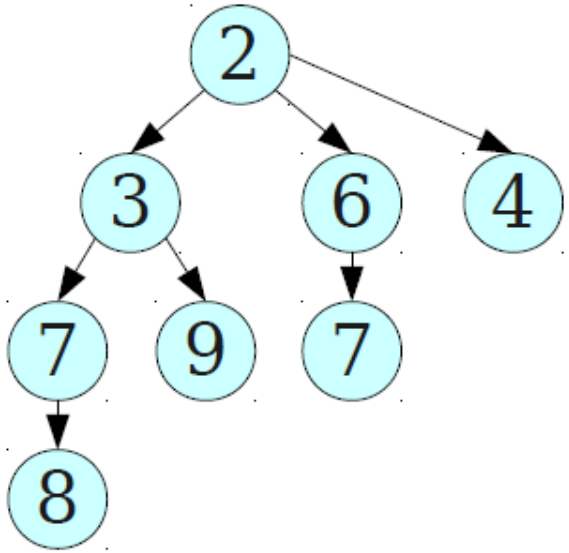
- Total time: $O(n^2)$

Or we can use priority queue to maintain active vertices

u is **complete**: d(u)=dist(s,u)

Heap-- Implementation of priority queue

- A **heap** is a specialized tree-based data structure that satisfies the *heap property:*
  - If A is a parent node of B then the key of node A is smaller than the key of node B

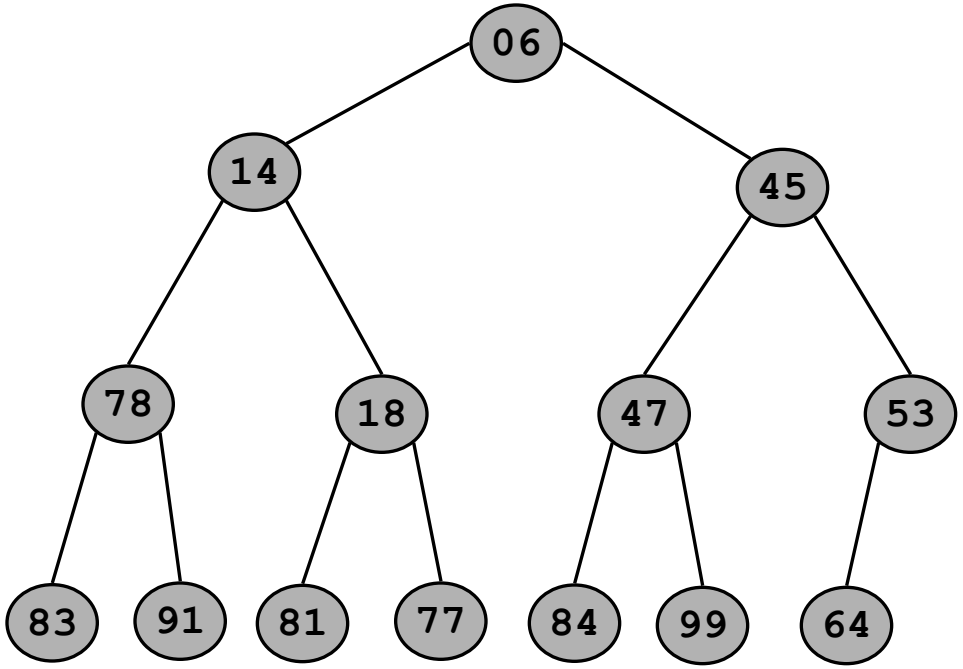




Binary Heap: Definition

Binary heap

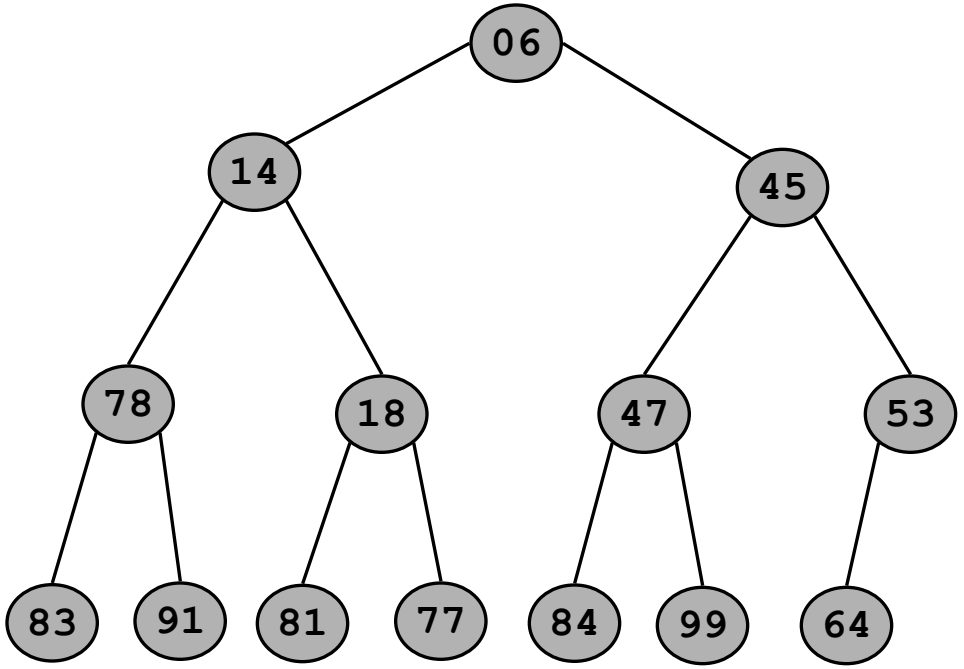
- Almost complete binary tree
  - filled on all levels, except last, where filled from left to right
- Min-heap ordered
  - every child greater than (or equal to) parent



Binary Heap: Properties

Properties

- Min element is in root
- Heap with n elements has height = $\lfloor \log_2 n \rfloor$

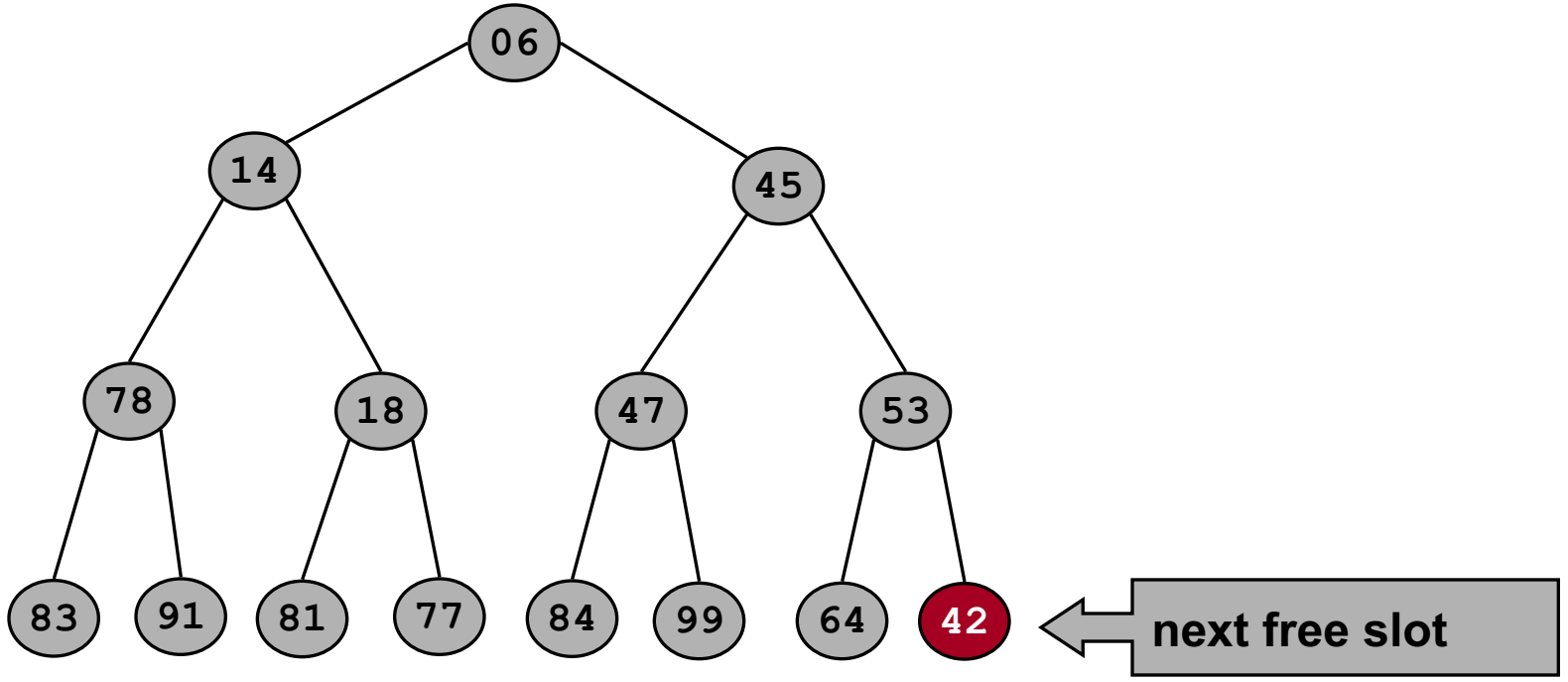


n = 14
Height = 3

Binary Heap: Insertion

Insert element x into heap

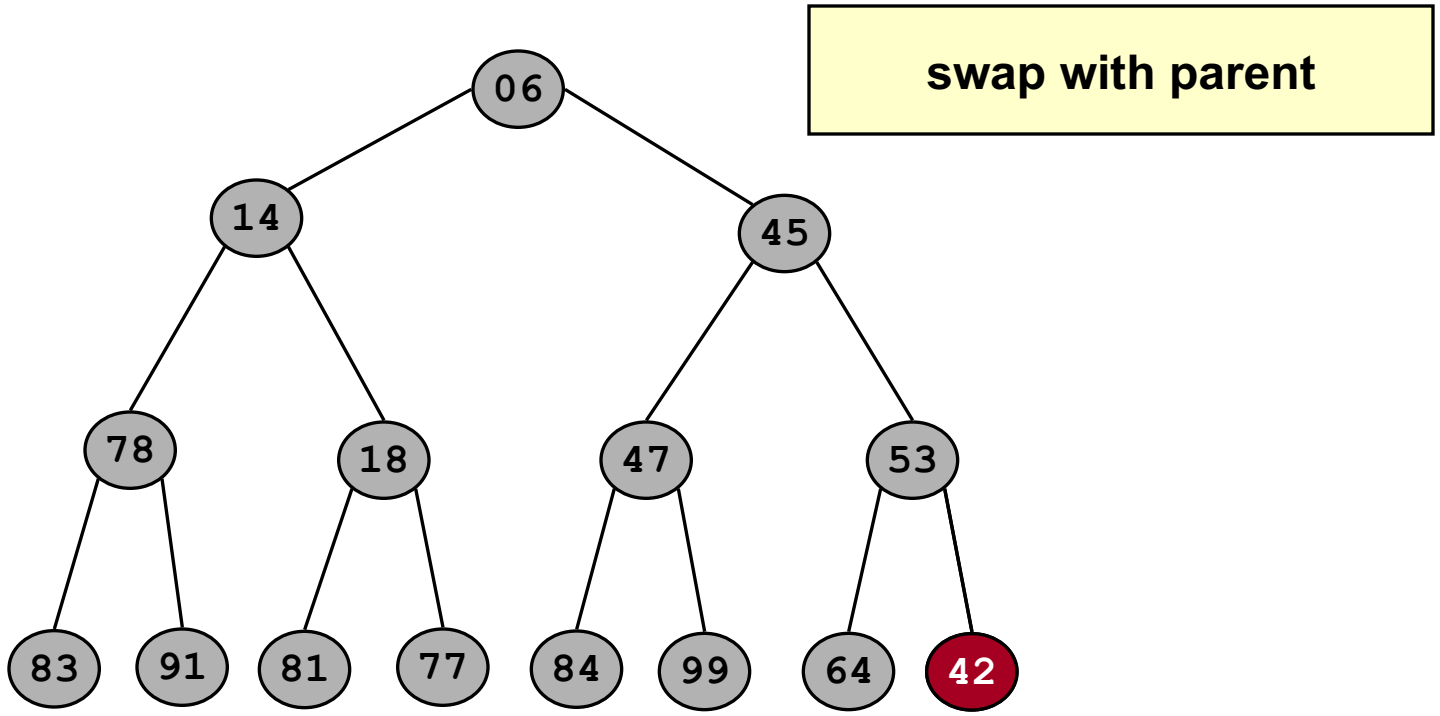
- Insert into next available slot
- Bubble up until it's heap ordered
  - Peter principle: nodes rise to level of incompetence



Binary Heap: Insertion

Insert element x into heap

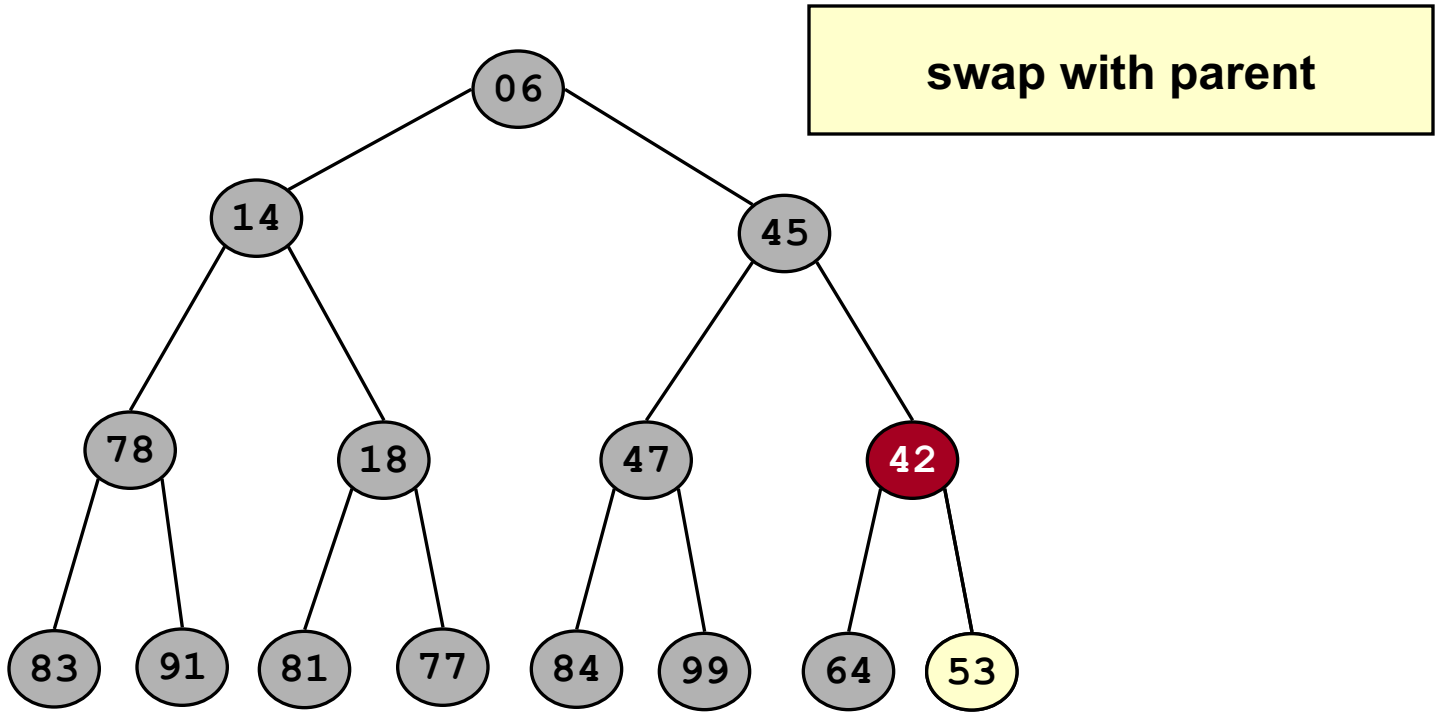
- Insert into next available slot
- Bubble up until it's heap ordered
  - Peter principle: nodes rise to level of incompetence



Binary Heap: Insertion

Insert element x into heap

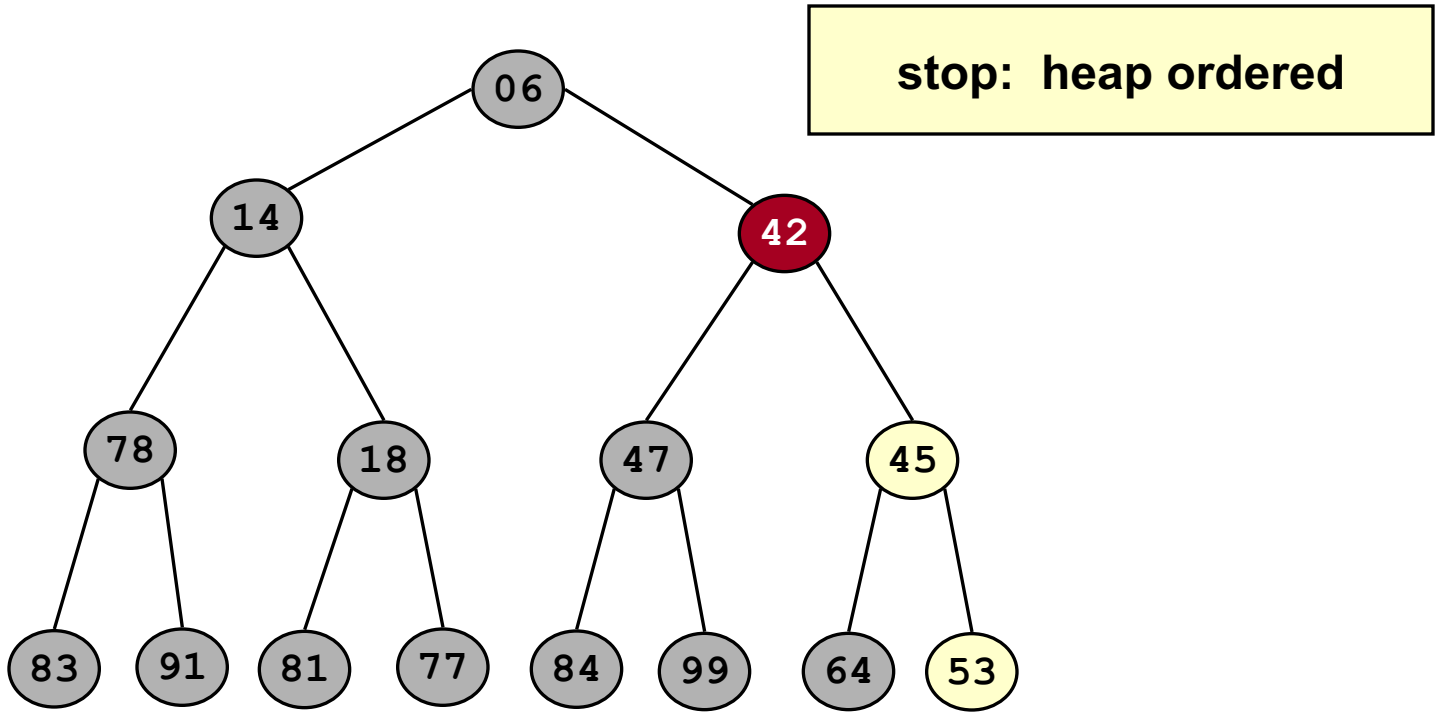
- Insert into next available slot
- Bubble up until it's heap ordered
  - Peter principle: nodes rise to level of incompetence



Binary Heap: Insertion

Insert element x into heap

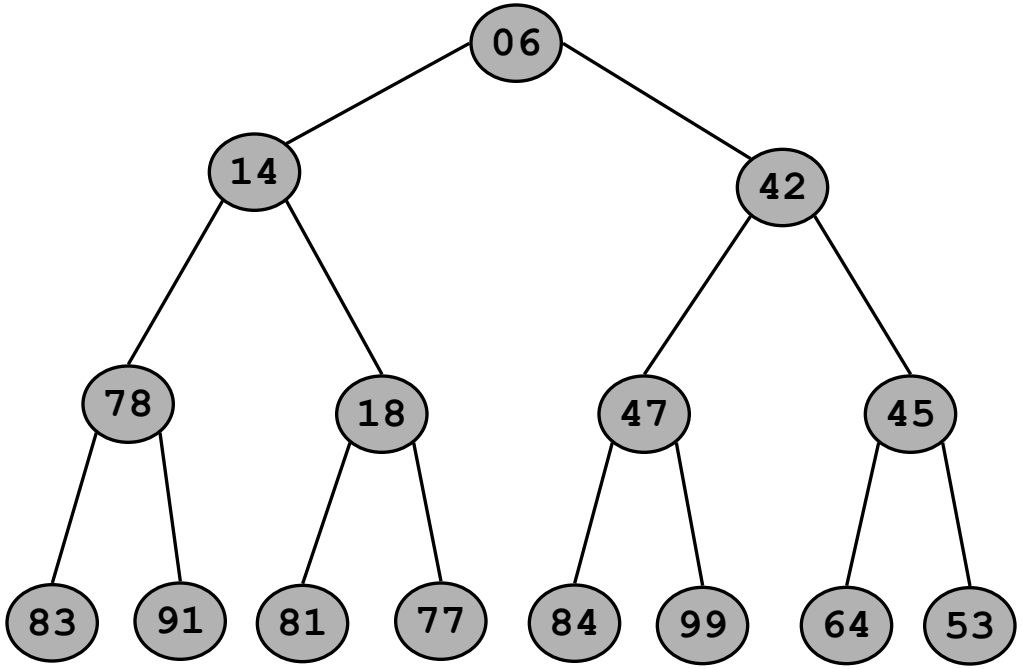
- Insert into next available slot
- Bubble up until it's heap ordered
  - Peter principle: nodes rise to level of incompetence
- O(log n) operations



Binary Heap: Decrease Key

Decrease key of element x to k

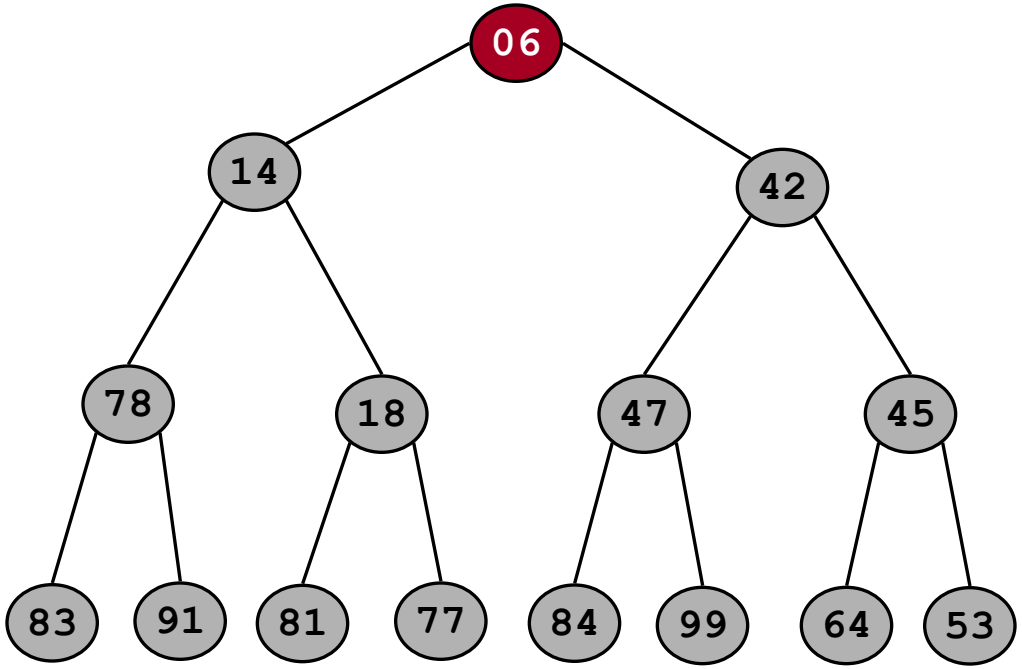
- Bubble up until it's heap ordered
- O(log n) operations



Binary Heap: Delete Min

Delete minimum element from heap

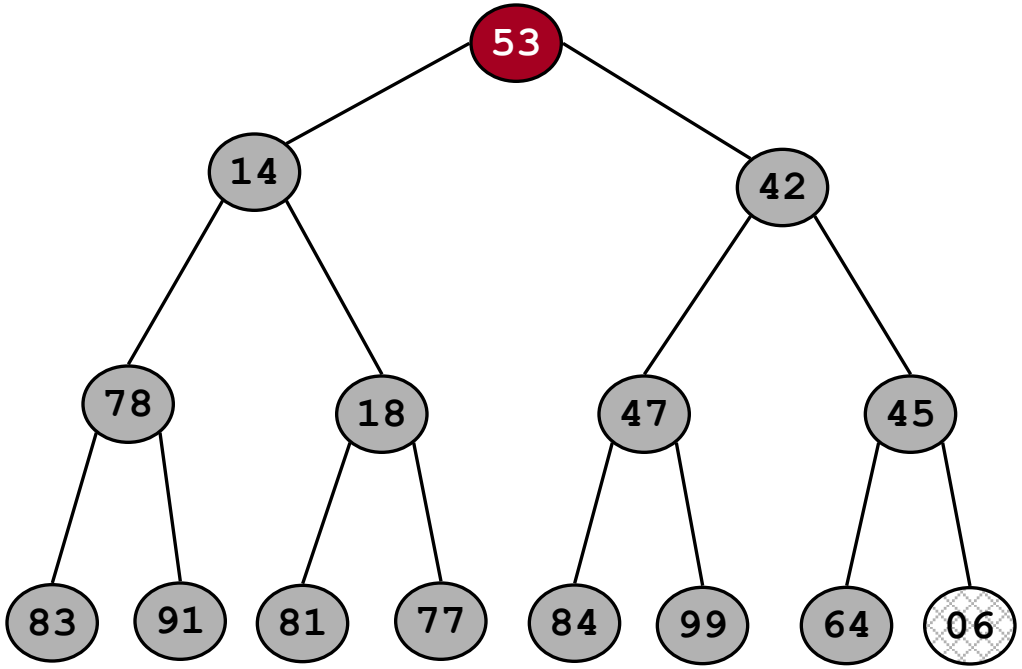
- Exchange root with rightmost leaf
- Bubble root down until it's heap ordered
  - power struggle principle: better subordinate is promoted



Binary Heap: Delete Min

Delete minimum element from heap

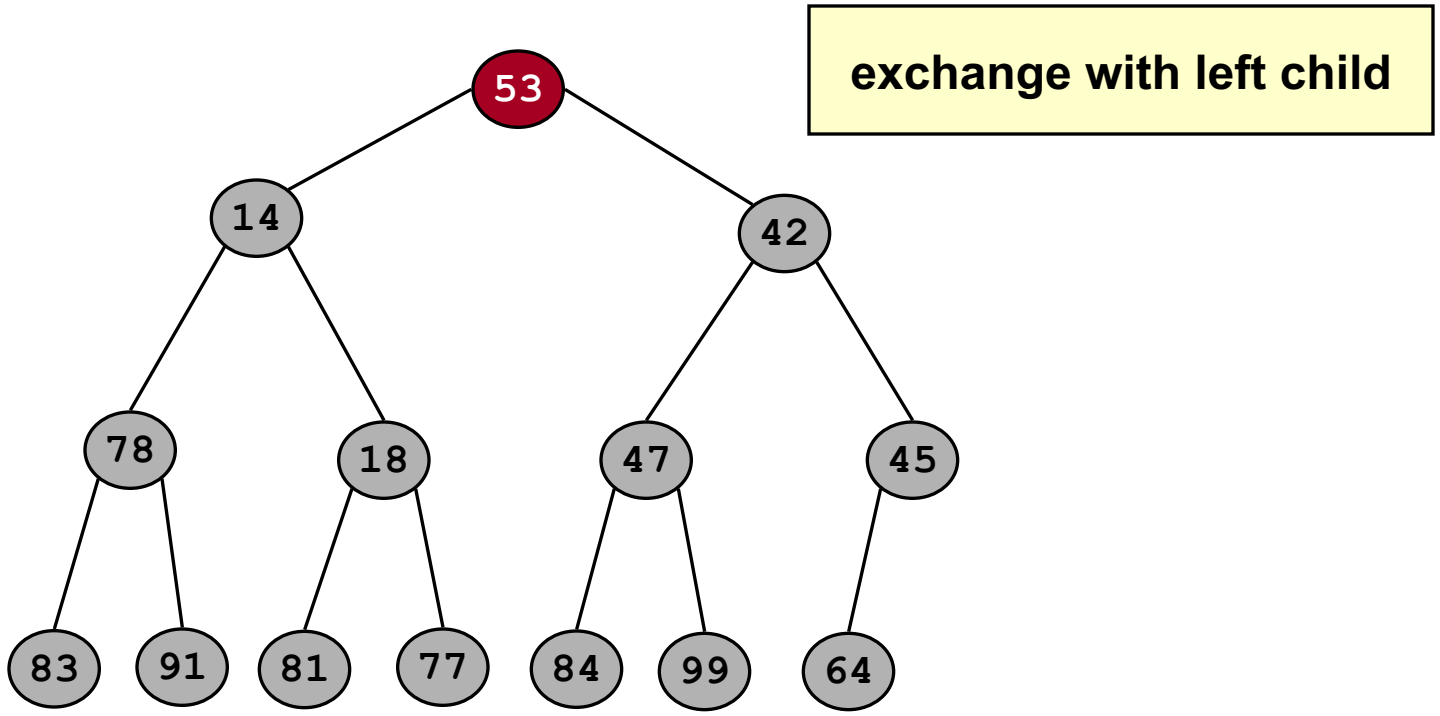
- Exchange root with rightmost leaf
- Bubble root down until it's heap ordered
  - power struggle principle: better subordinate is promoted



Binary Heap: Delete Min

Delete minimum element from heap

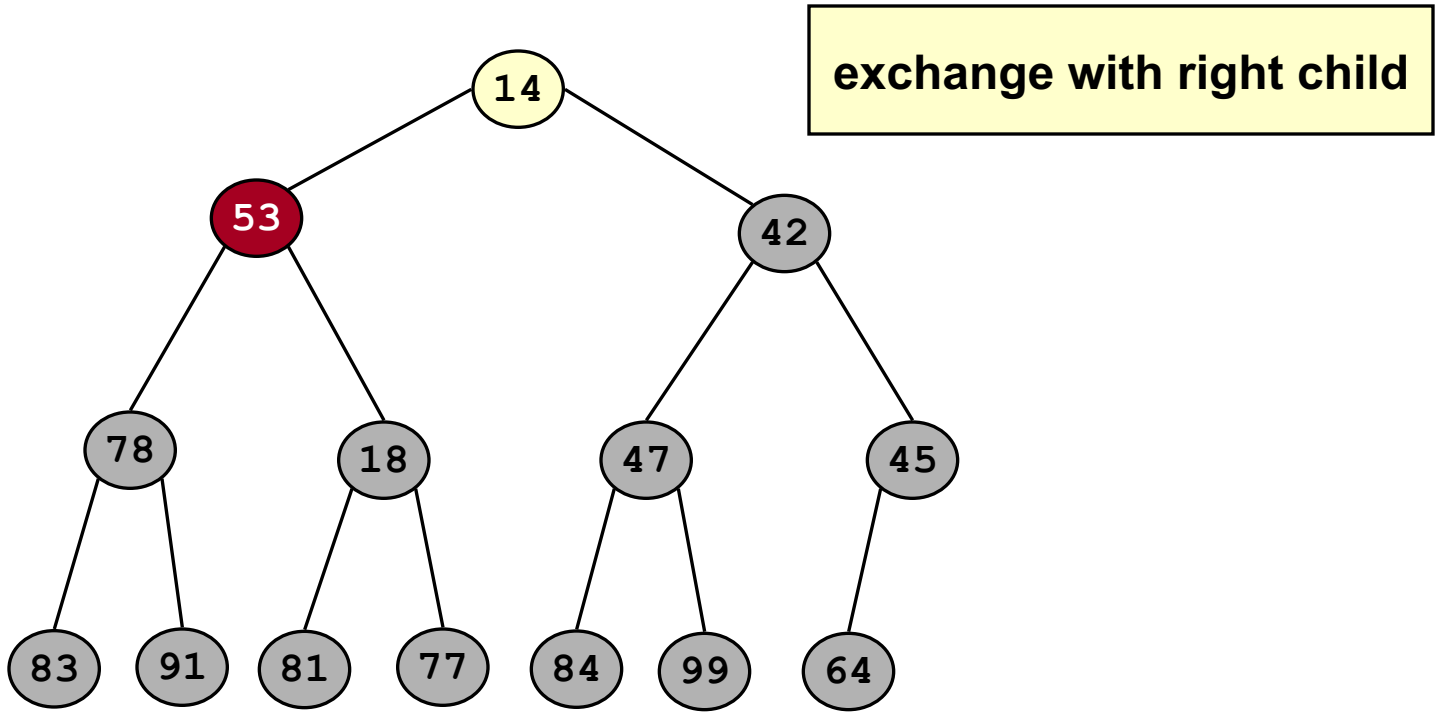
- Exchange root with rightmost leaf
- Bubble root down until it's heap ordered
  - power struggle principle: better subordinate is promoted



Binary Heap: Delete Min

Delete minimum element from heap

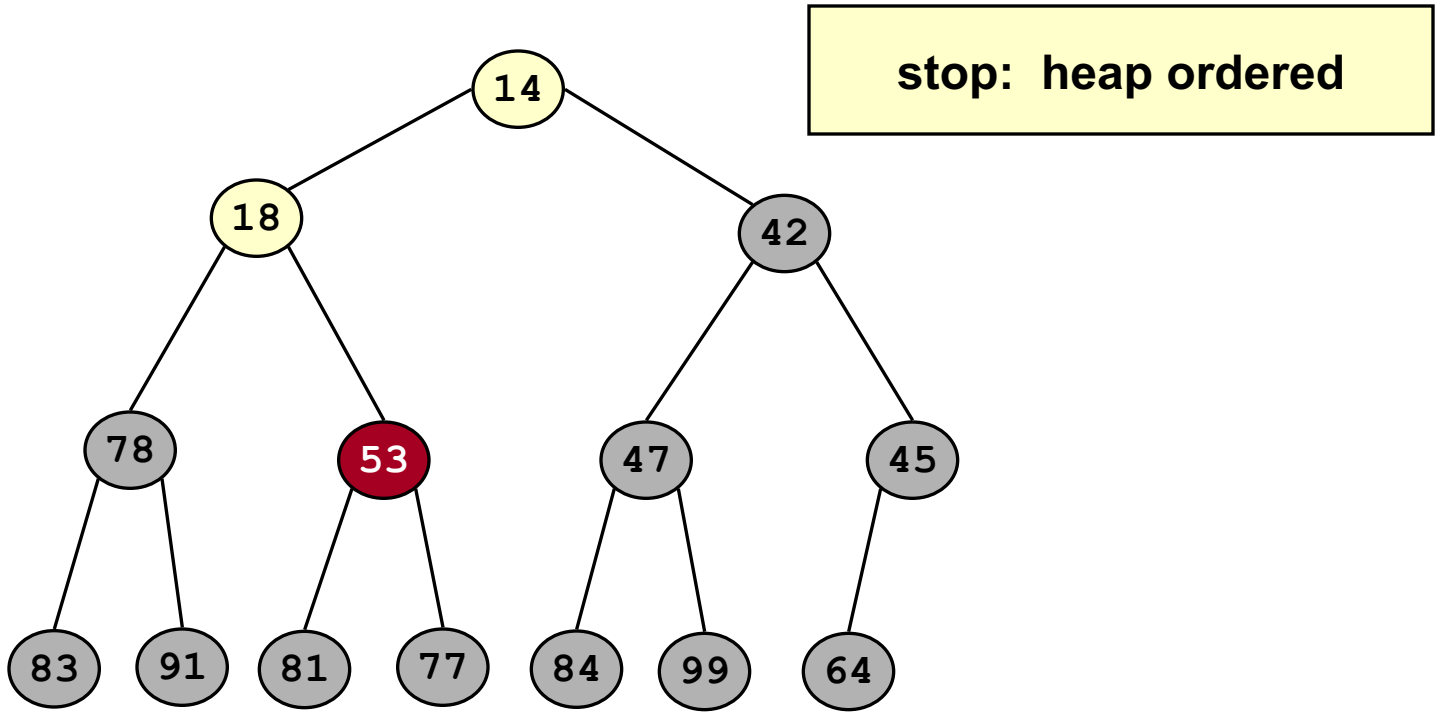
- Exchange root with rightmost leaf
- Bubble root down until it's heap ordered
  - power struggle principle: better subordinate is promoted



Binary Heap: Delete Min

Delete minimum element from heap

- Exchange root with rightmost leaf
- Bubble root down until it's heap ordered
  - power struggle principle: better subordinate is promoted
- O(log n) operations



Dijkstra's algorithm-Priority queues

```
Label: d(s)←0, d(v)←+∞ for all other v
Mark all vertices active
While there is an active vertex
    Select an active u with minimum d(u)
    mark u complete
    Relax every outgoing edge (u,v):
        d(v)← min{d(v), d(u)+w(u,v)}
```

Insert

Delete minimum

Decrease key

n insert, n delete-min, O(m) decrease-key

binary heap: O(m·log n)
Fibonacci heap: O(m+n·log n)

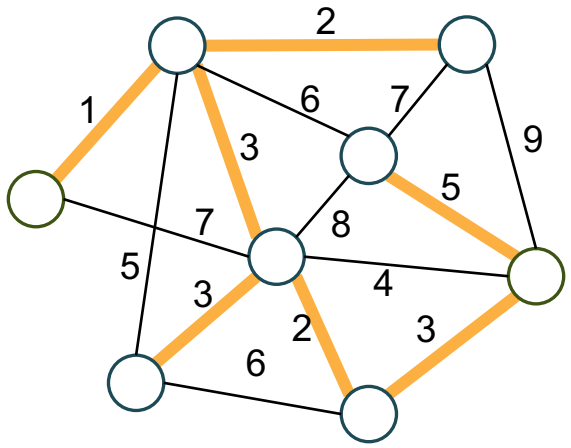
Operations	Binary heap	Binomial heap	Fibonacci heap †
insert	log N	log N	1
delete-min	log N	log N	log N
decrease-key	log N	log N	1

† amortized time

Related problems

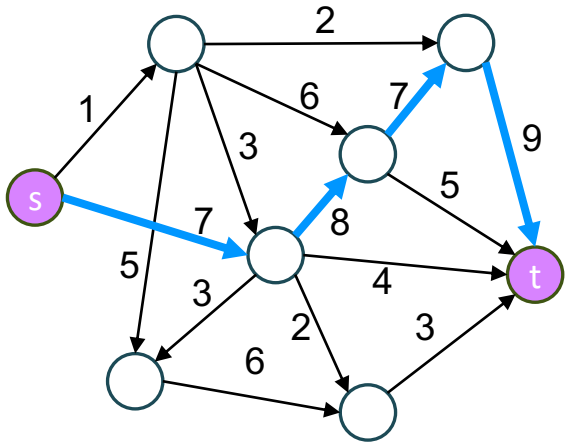
Variants of Dijkstra's algorithm of O(m+n·log n) time also works for other problems:

Minimum Spanning Tree



Tree connecting all vertices with minimum total edge weights

Bottleneck Path/Widest Path



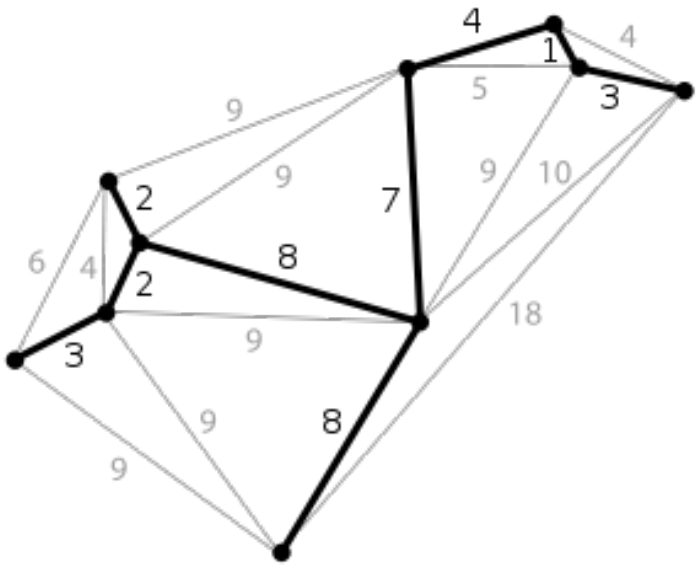
maximizing the weight of the minimum-weight edge in the path

Minimum Spanning Tree

Given a **connected, undirected** graph G=(V,E) with real weight edges w

Spanning tree: a tree that connects all vertices

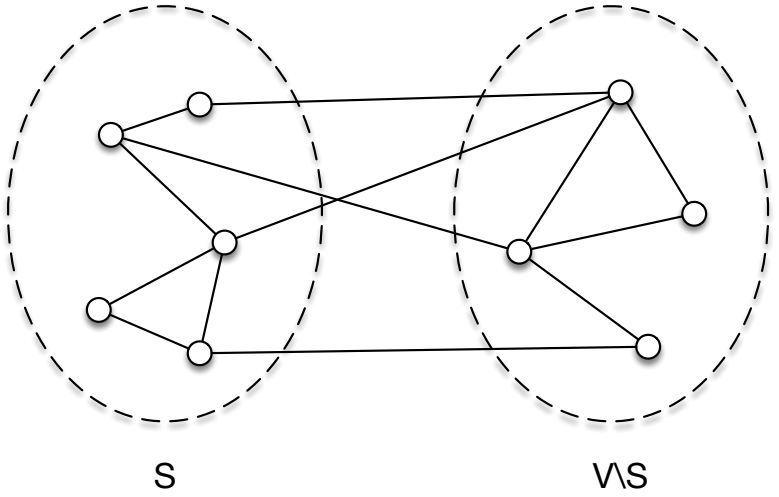
Minimum spanning tree: the spanning tree with minimum total weight



Cut (in undirected graphs)

In graph theory, a **cut** is a partition of the vertices of a graph into two disjoint subsets:

- For any $S \subseteq V$, $(S, V/S)$ is a cut

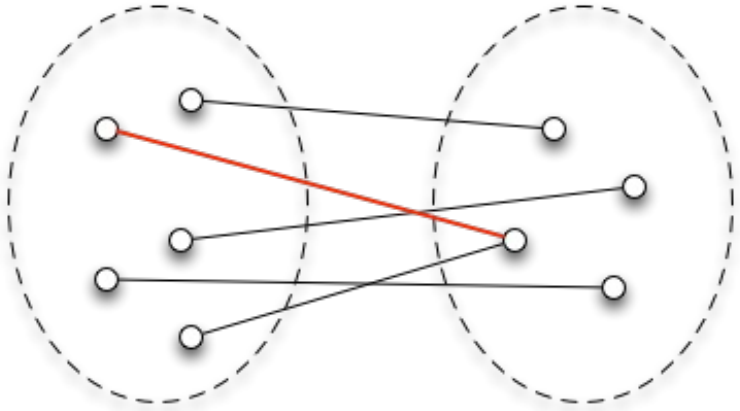


We assume all edge weights are distinct for simplicity

Two basic properties for MST

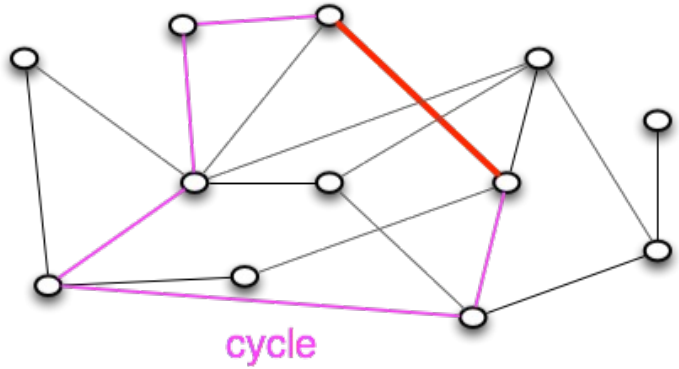
CUT PROPERTY

(One of) The lightest edge crossing any cut of G must be in the MST



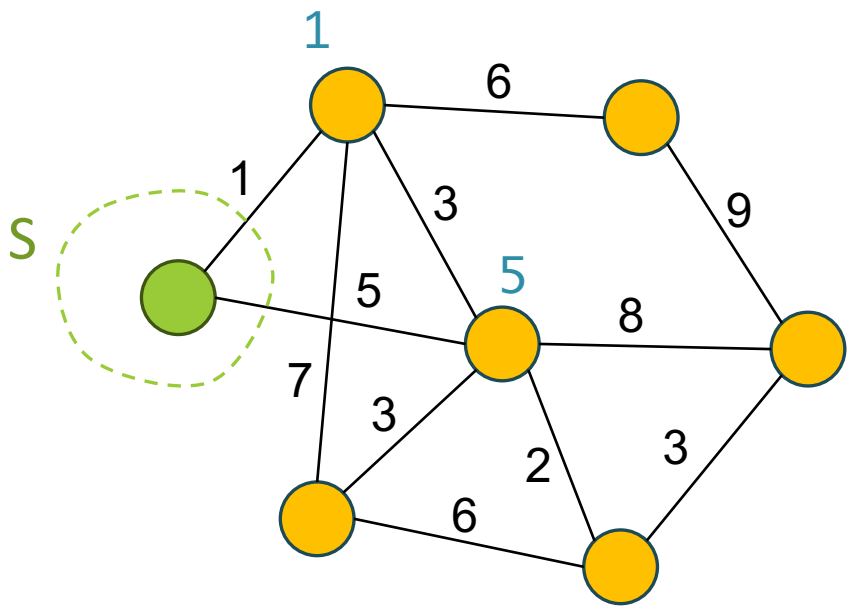
CYCLE PROPERTY

(One of) The heaviest edge in any cycle in G cannot be in the MST



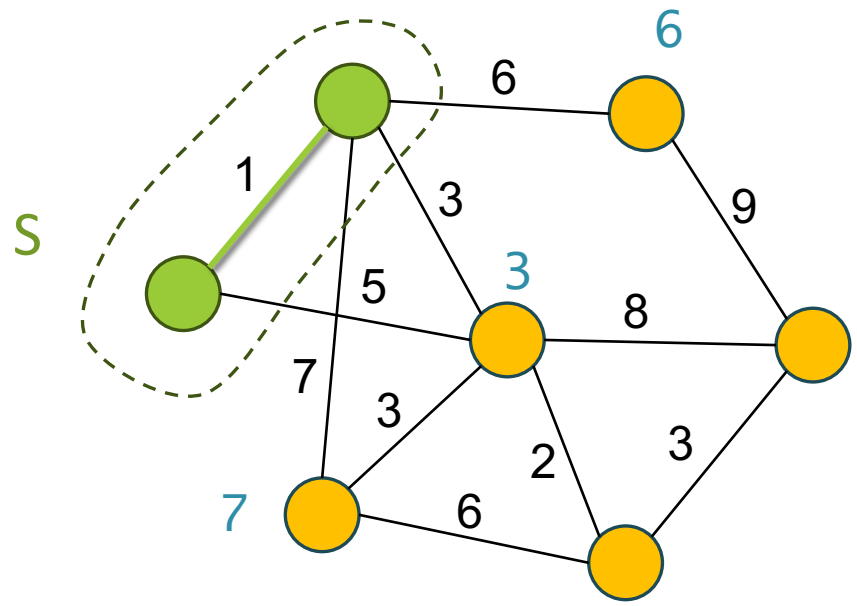
Minimum Spanning Tree

The lightest edge crossing any cut of G must be in the MST



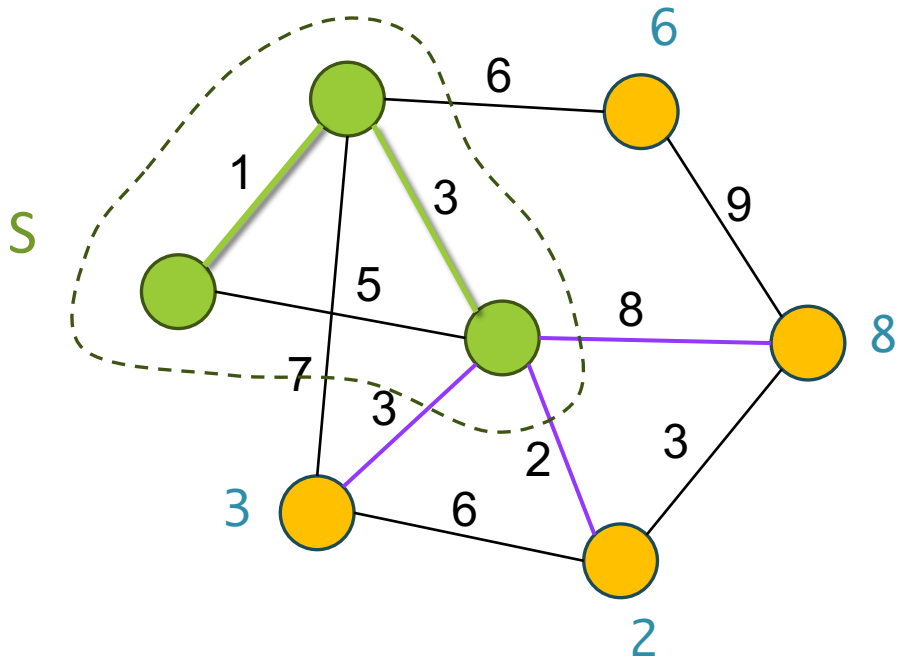
Minimum Spanning Tree

The lightest edge crossing any cut of G must be in the MST



Minimum Spanning Tree

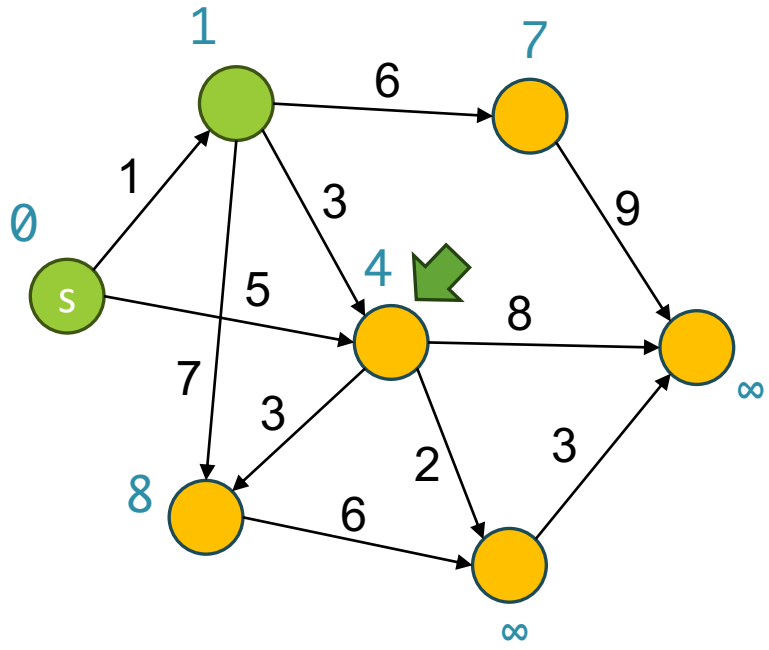
The lightest edge crossing any cut of G must be in the MST



d(v): lightest edge from S to v

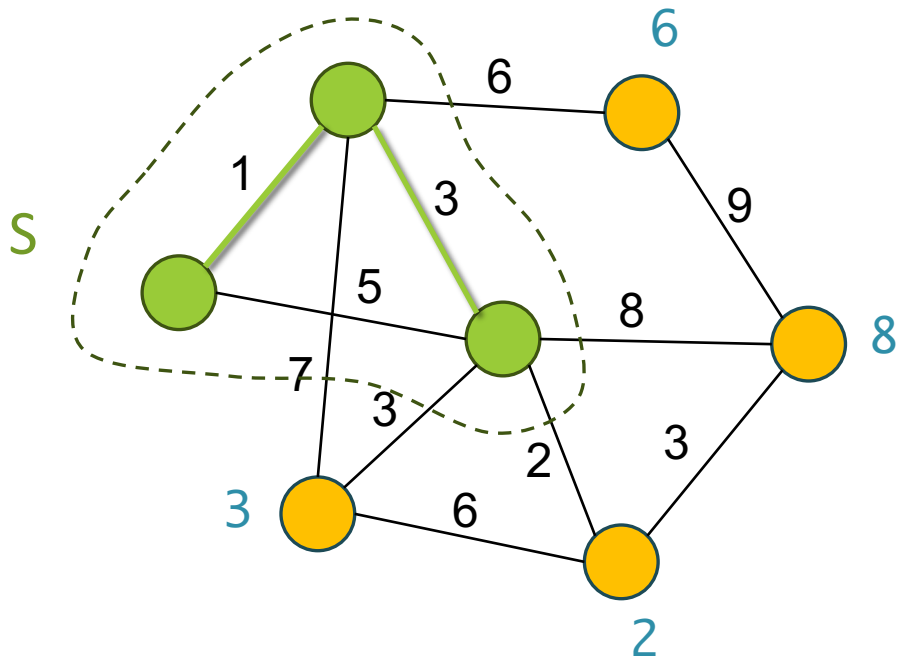
Dijkstra's algorithm

```
Label: d(s)←0, d(v)←+∞ for all other v
Mark all vertices active
While there is an active vertex
    Select an active u with minimum d(u)
    mark u complete
    Relax every outgoing edge (u,v):
        d(v)← min{d(v), d(u)+w(u,v)}
```



Prim's algorithm

```
Label: d(s)←0, d(v)←+∞ for all other v
Mark all vertices active
While there is an active vertex
    Select an active u with minimum d(u)
        (Put the (u,v) with w(u,v)=d(v) in the spanning tree)
    mark u complete
    Relax every outgoing edge (u,v):
        d(v)← min{d(v), w(u,v)}
```

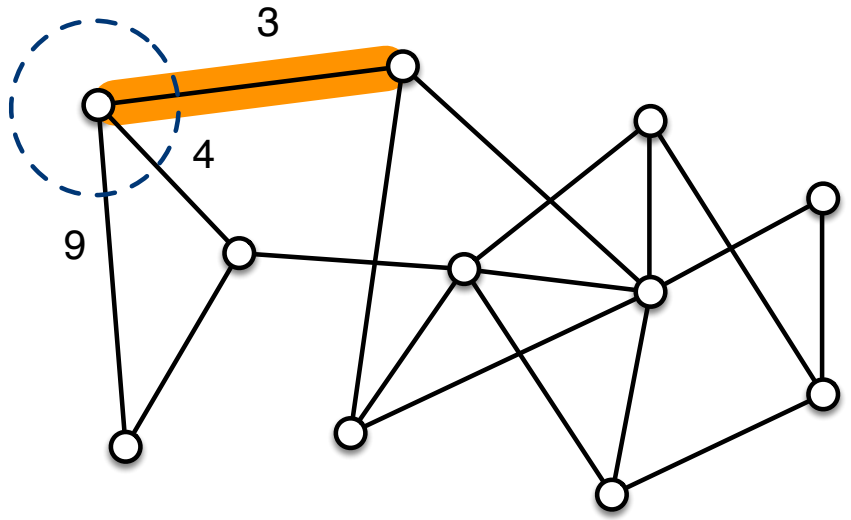


Robert Prim published it in *Bell System Technical Journal 1957*
Edsger Dijkstra published it in *Numerische Mathematik 1959*
Later, people found that Vojtěch Jarník has published it as *"O jistém problému minimálním" (in Czech)* in 1930

So, sometimes called Jarník's algorithm or DJP algorithm

Borůvka's algorithm

For each vertex, its lightest edge must be in the MST

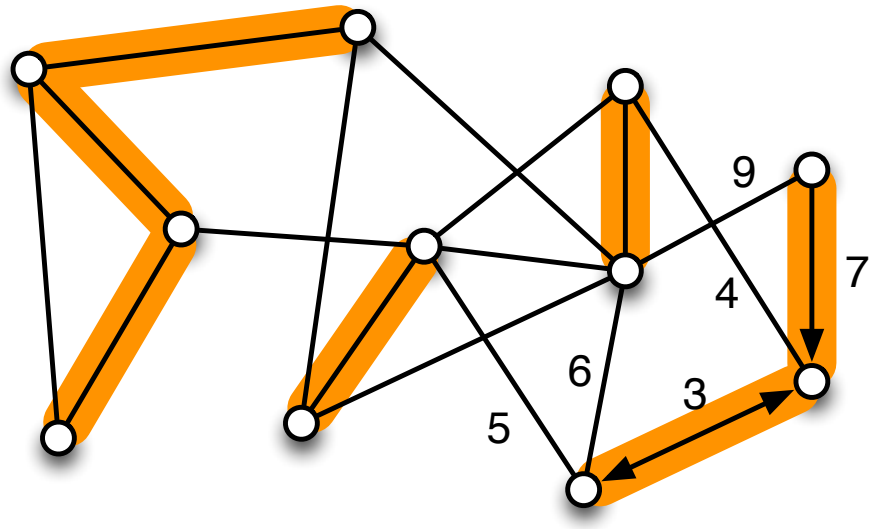


Borůvka's algorithm

For each vertex, add its lightest edge to the tree T

- lightest edges of two vertices may be the same one

Contract each connected components

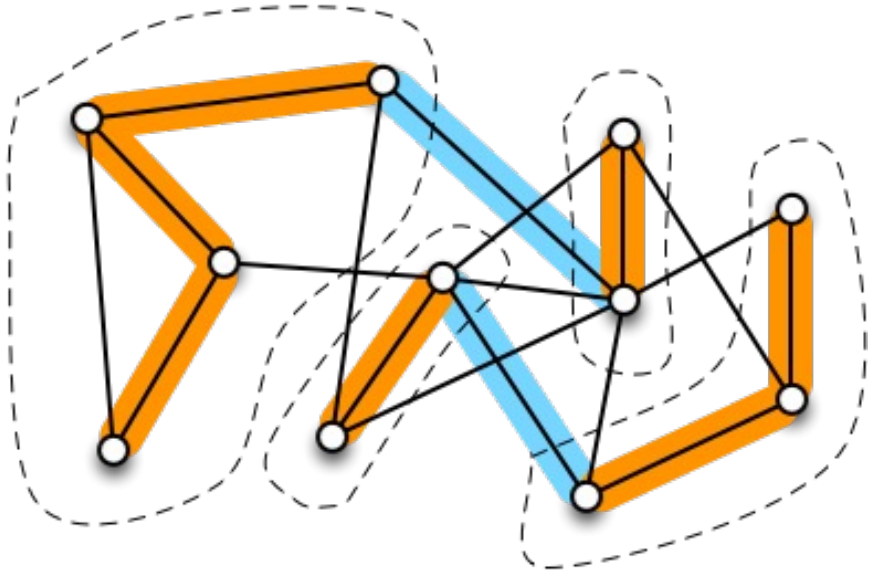


Borůvka's algorithm

For each vertex, add its lightest edge to the tree T

Contract each connected components

Recursively repeat this until the whole graph is contracted into a single vertex



Borůvka's algorithm

For each vertex, add its lightest edge to the tree T

Contract each connected components

Recursively repeat this until the whole graph is contracted into a single vertex

- so the number of vertices shrink by at least one half in each iteration

Running time: Each iteration takes $O(m)$ time, and there are $O(\log n)$ iterations, so the time is $O(m \cdot \log n)$

Better algorithm than O(m·log n)?

O(m·loglog n) time algorithm *[Yao 1975]*

- *Yao cited an unpublished result of Tarjan with O(m log$^{1/2}$n) time*

AN O(|E| log log |V|) ALGORITHM FOR FINDING MINIMUM SPANNING TREES *

Andrew Chi-chih YAO
Department of Computer Science, University of Illinois, Urbana, Illinois 61801, USA

Received 30 December 1975, revised version received 9 June 1975

Minimum spanning tree, linear median finding algorithm

1. Introduction

Given a connected, undirected graph $G = (V, E)$ and a function c which assigns a cost $c(e)$ to every edge $e \in E$, it is desired to find a spanning tree T for G such that $\Sigma_{e \in T} c(e)$ is minimal. In this note we describe an algorithm which finds a minimum spanning tree (MST) in $O(|E| \log \log |V|)$ time. Previously the best MST algorithms known have running time $O(|E| \times \log |V|)$ for sparse graphs [1], and more recently

plying the linear median-finding algorithm [4]. Having accomplished this, we follow basically Sollin's algorithm as outlined above. Note that the number of operations needed in this phase is now reduced to

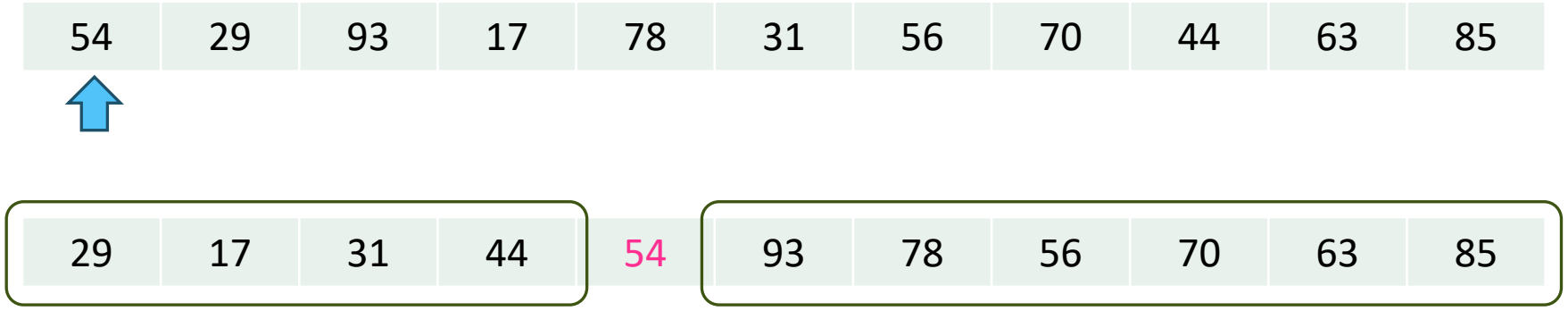
$$O\left(\frac{|E|}{k} \log |V|\right)$$

since only approximately $|E|/k$ edges have to be examined at each stage to find the minimum-cost edges incident with all the nodes. Therefore, the total number

Quicksort

Every time randomly select an element and use it to partition the list into two intervals

Recursively deal with each interval



Median-find algorithm

We can find the median of a list in linear time O(N)

Partition the list into two intervals: smaller than median and larger than median

Recursively do ℓ levels

- partition the list into 2^{ℓ} intervals
- time: O(ℓN) time

54	29	93	17	78	31	56	70	44	63	85
54	29	17	31	44	56	93	78	70	63	85
29	17	31	54	44	56	70	63	78	93	85

Median-find algorithm

We can partition the list into k intervals by order in O(N·log k) time

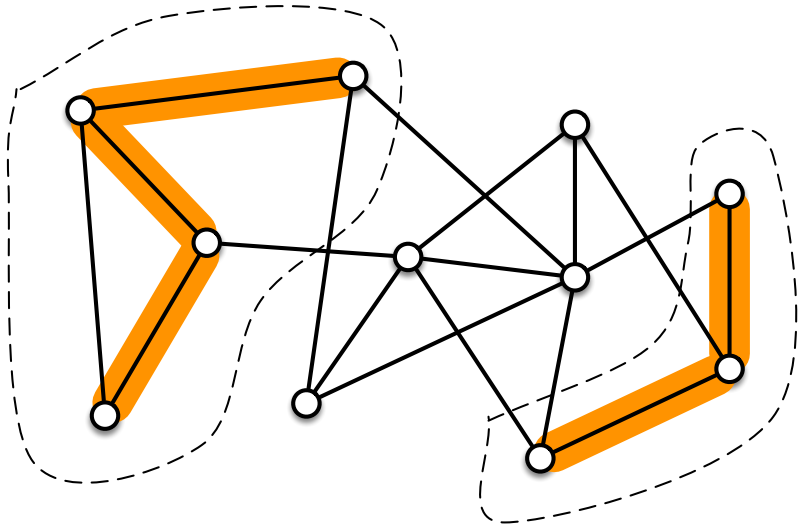
54	29	93	17	78	31	56	70	44	63	85
54	29	17	31	44	56	93	78	70	63	85
29	17	31	54	44	56	70	63	78	93	85

[Yao 1975] for MST

Run **Borůvka's step** for vertices of degree ≤ log n

Until all vertices have degree > log n

O(loglog n) iterations of **Borůvka's steps**, so O(m·loglog n) time

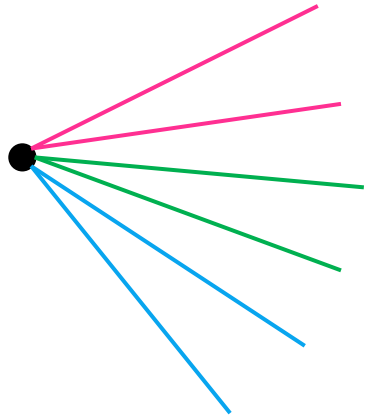


[Yao 1975] for MST

All vertices have degree > log n

Partition the edges associated with each vertex into k intervals

- in total O(m·log k) time
- if we need to find the minimum among M edges, now only need O(M/k) time

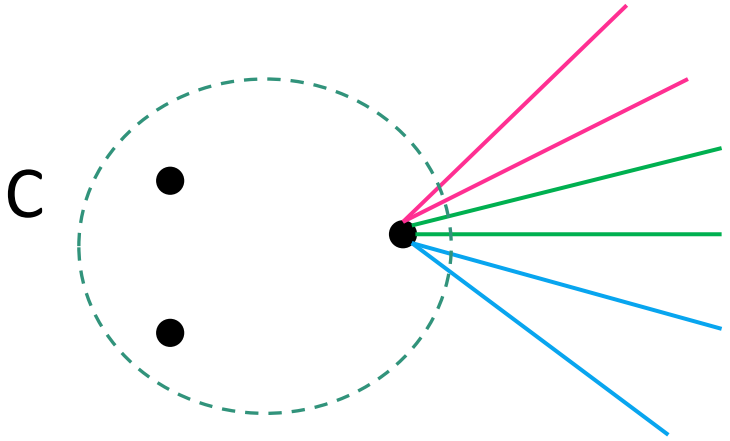


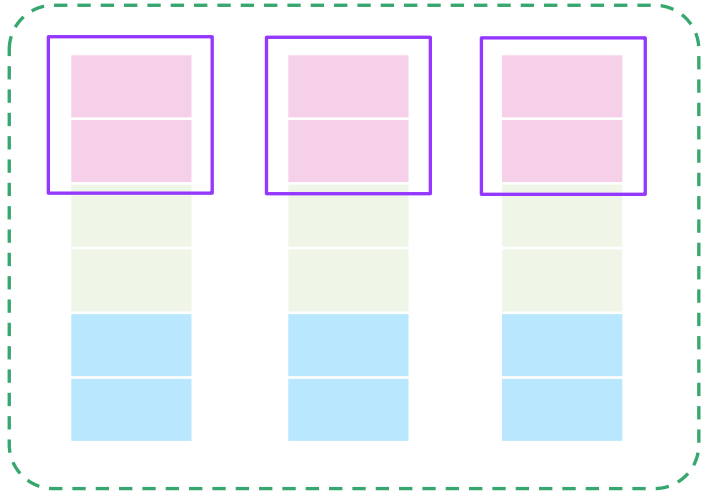
[Yao 1975] for MST

All vertices have degree > log n

Partition the edges associated with each vertex into k intervals

- in total O(m·log k) time
- After shrinking, we need to find the minimum among all edges associated with this component C
- Check edges from lightest intervals of vertices in C



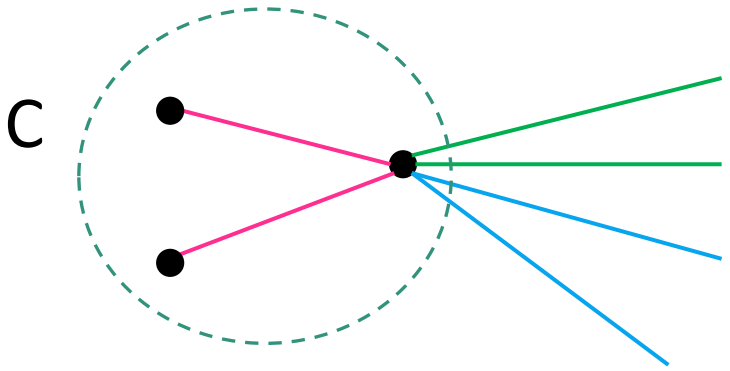


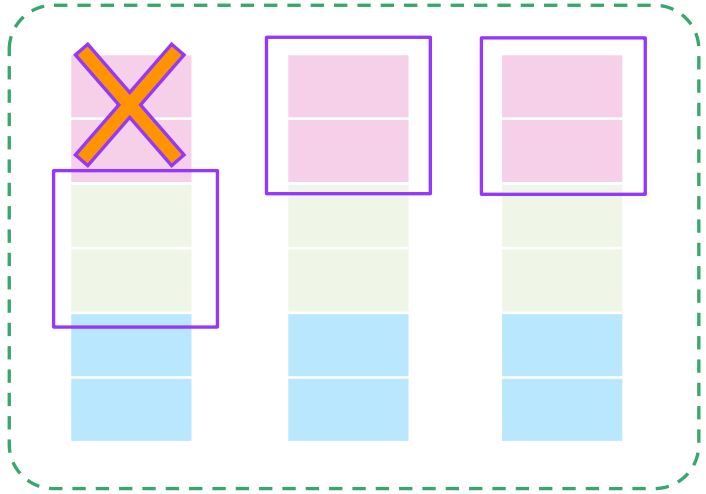
[Yao 1975] for MST

All vertices have degree > log n

Partition the edges associated with each vertex into k intervals

- in total O(m·log k) time
- After shrinking, we need to find the minimum among all edges associated with this component C
- Check edges from lightest intervals of vertices in C
- Edges can be within C, delete them





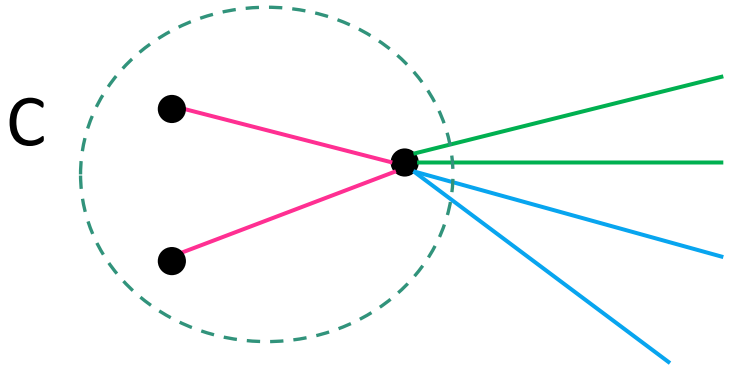
[Yao 1975] for MST

All vertices have degree > log n, so $m > n \cdot \log n$

Partition the edges associated with each vertex into k intervals

Inside a component C

- for each vertex $v \in C$, find the lightest edge (v,w) of v so that $w \notin C$
- delete edges within C



TIME ANALYSIS
partition: O(m·log k)
successful finding lightest edges: O(m/k · log n)
unsuccessful finding (deletion): O(m)
set union: O(n·log n)

Choose k=log n, the total time is O(m·loglog n)

Best algorithms for MST

Algorithms:

- Randomized algorithm with linear time O(m). *[Karger, Klein, Tarjan 1995]*
- Deterministic algorithm with O(mα(m,n)) time *[Chazelle 2000]*
- Deterministic algorithm with proved optimal running time by decision trees *[Pettie, Ramachandran 2002]*

Verification:

- Verify a MST in linear time O(m). *[Dixon, Rauch, Tarjan 1992]*

α(m,n) is the inverse of the Ackermann function, grows **extremely** slowly

Bottleneck paths

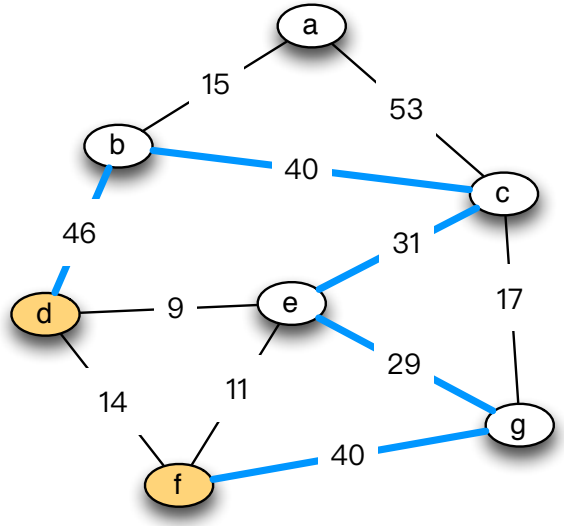
s-t bottleneck path: the path with maximum flow from *s* to *t*

flow of a path: the minimum capacity of edges on that path

For a graph *G=(V,E),* and the capacity function for every edge: *w: E→*$\mathbb{R}$, the bottleneck path is defined as:

$$f(s,t) = \max_{p:path \quad s \to t} \min_{e \in p} w(e)$$

called “widest path” in wikipedia



Bottleneck paths

s-t bottleneck path: the path with maximum flow from *s* to *t*

flow of a path: the minimum capacity of edges on that path

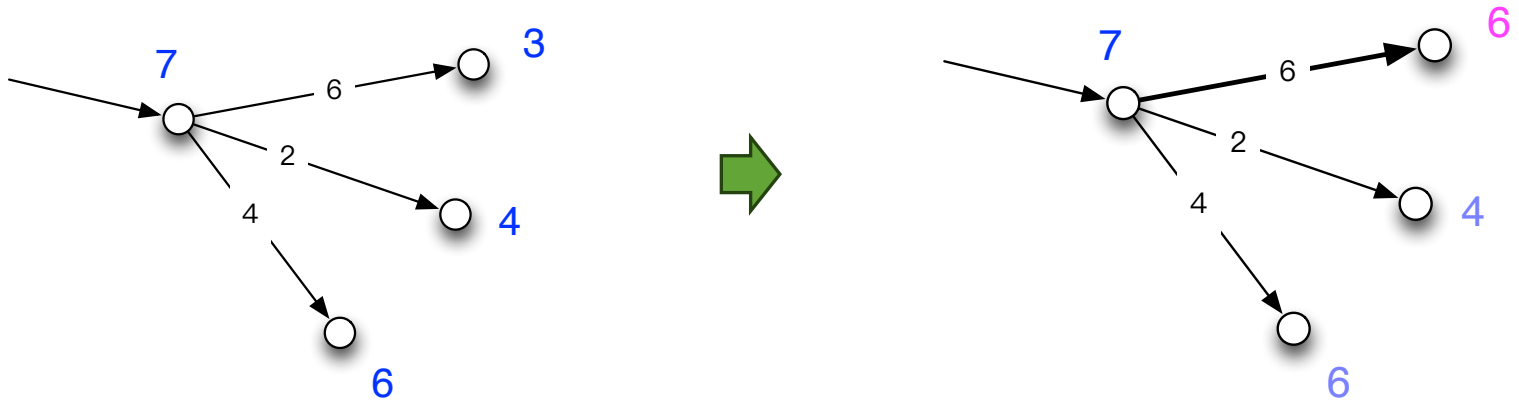
For a graph *G=(V,E),* and the capacity function for every edge: *w: E→*$\mathbb{R}$, the bottleneck path is defined as:

$$f(s,t) = \max_{p:path} \min_{s \to t} \min_{e \in p} w(e)$$

It's also like: *s can reach t* only using edges of weights ≥k, and maximize k

Dijkstra's algorithm for bottleneck path

```
Label: d(s)←+∞, d(v)←0 for all other v
Mark all vertices active
While there is an active vertex
    Select an active u with maximum d(u)
    mark u complete
    Relax every outgoing edge (u,v):
        d(v)← max{d(v), min{d(u),w(u,v)}}
```



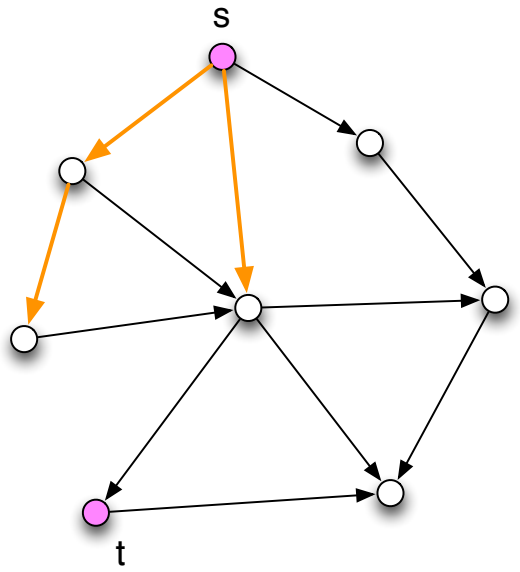
Gabow & Tarjan's s-t bottleneck path

Partition all edges into k intervals (in decreasing order)

Takes O(m log k) time

Then we run BFS from s using:

- initially edges in the largest interval
- adding more and more intervals
- until we reach t





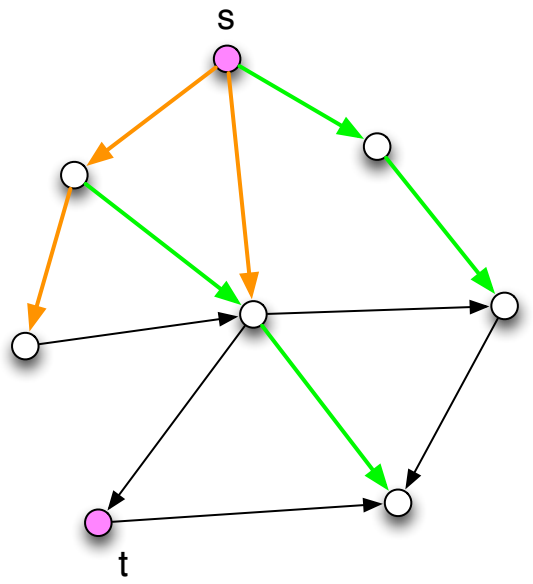
Gabow & Tarjan's s-t bottleneck path

Partition all edges into k intervals (in decreasing order)

Takes O(m log k) time

Then we run BFS from s using:

- initially edges in the largest interval
- adding more and more intervals
- until we reach t





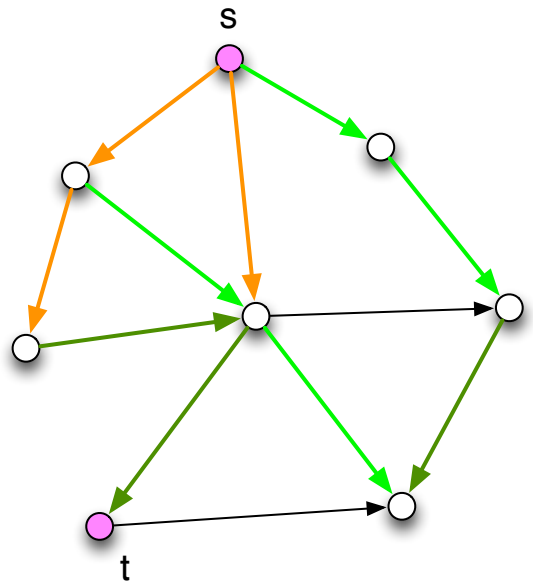
Gabow & Tarjan's s-t bottleneck path

Partition all edges into k intervals (in decreasing order)

Takes O(m log k) time

Then we run BFS from s using:

- initially edges in the largest interval
- adding more and more intervals
- until we reach t





Gabow & Tarjan's s-t bottleneck path

Partition all edges into k intervals (in decreasing order)

So in O(m log k) time we can find the s-t bottleneck path is in which interval

then recursion on such interval

- if that interval has m' edges, partitioning into k' intervals takes O(m'log k') time
- BFS to find s-t bottleneck path is in which interval takes O(m) time
- since m'=m/k, we can choose k'=2^k to make O(m'log k')=O(m)

- in third step, the interval has m''=m'/k'≤m/2^k edges, partition into k'' intervals, and the time will be

  O(m''·log k''+m)

- if $k'' = 2^{2^k}$ it is O(m)



Gabow & Tarjan's s-t bottleneck path

Partition all edges into k intervals (in decreasing order)

So in O(m log k) time we can find the s-t bottleneck path is in which interval

then recursion on such interval

- if that interval has m' edges, partitioning into k' interval takes O(m'log k') time
- BFS to find s-t bottleneck path is in which interval takes O(m) time
- since m'=m/k, we can choose $k'=2^k$ to make O(m'log k')=O(m)

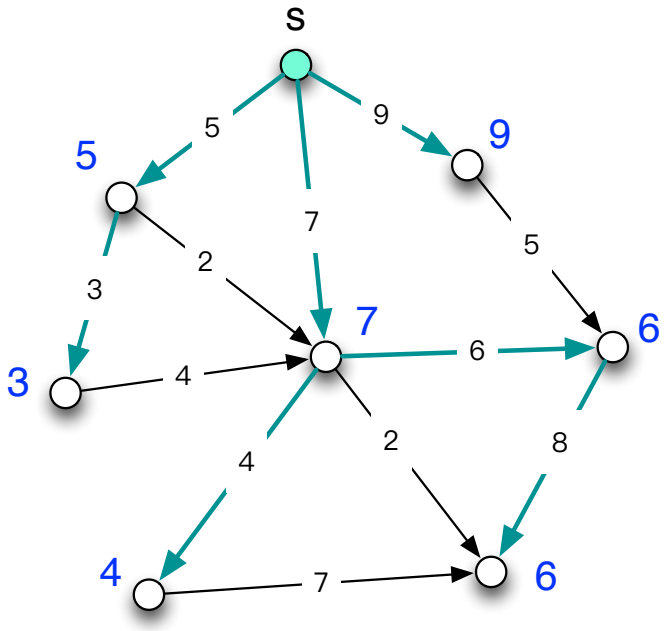
in i-th step, let $k_i=2^{2^{2^{\cdots 2^2}}}$ (i times), the total running time is $O(m\cdot\log^* n)$

log*n is the number of times the logarithm function must be iteratively applied before the result is less than or equal to 1



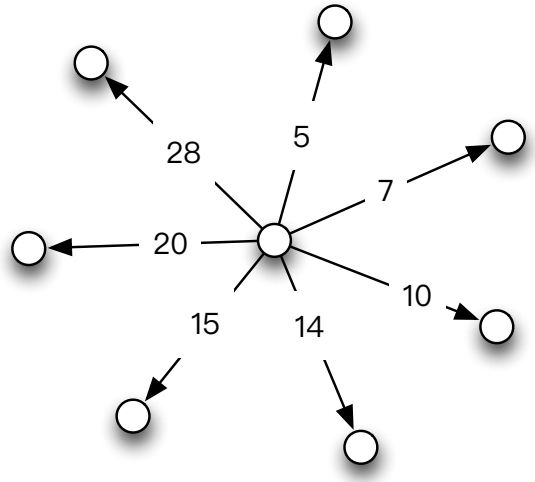
Single source bottleneck paths (SSBP)

SSBP: given a source node s, find the bottleneck paths from s to all other vertices



Dijkstra's algorithm -- SSBP

```
Label: d(s)←+∞, d(v)←0 for all other v
Mark all vertices active
While there is an active vertex
    Select an active u with maximum d(u)
    mark u complete
    Relax every outgoing edge (u,v):
        d(v)← max{d(v), min{d(u),w(u,v)}}
```



Dijkstra's algorithm by Fibonacci heap: O(m+n·log n)

- better time for sparse graphs?

Dijkstra's algorithm sorts vertices by their distances from s

- In *comparison* model, sorting takes Θ(n·log n) time
- To break Θ(n·log n) time bound for sparse graphs, we need to avoid sorting all vertices

Bottleneck Path Algorithms (directed graphs)

(Comparison model: we can only compare edge weights)

Dijkstra's approach by Fibonacci heap: $O(m+n \cdot \log n)$

s-t BP:

- deterministic $O(m \cdot \log^* n)$ *[Gabow & Tarjan 1988]*
- randomized $O(m \cdot \beta(m,n))$ *[Chechik et al. 2016]*

All-pair bottleneck path:

- $O(n^{(3+\omega)/2}) \approx O(n^{2.688})$ *[Duan & Pettie 2009]*

Single-source bottleneck path:

- $O(m\sqrt{\log n})$ *[Duan, Lyu, Wu & Xie 2018]*
- first result to break the sorting barrier

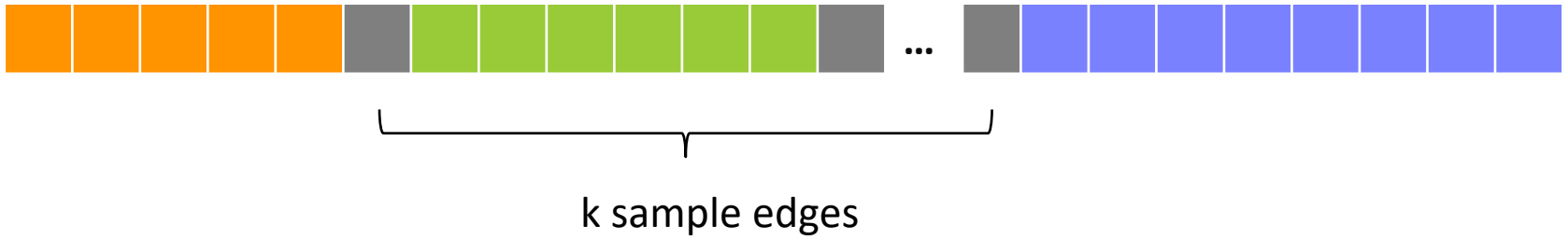
Random sampling

Inspired by *[Chechik et al. 2016]*, instead of sorting all edges, we randomly select k edges and sort them

O(k log k) time

then it will partition all edges into (k+1) intervals by order

Locating an edge in which interval takes O(log k) time



Random sampling

Run Dijkstra algorithm in which d’ and **L(w(u,v))** are intervals

For every outgoing edge (u,v)
d’(v)← max{d’(v), min{d’(u), L(w(u,v))}}

L(w(u,v)): interval for w(u,v)

Finally **d’(v)** indicate the s-v bottleneck path is in which interval

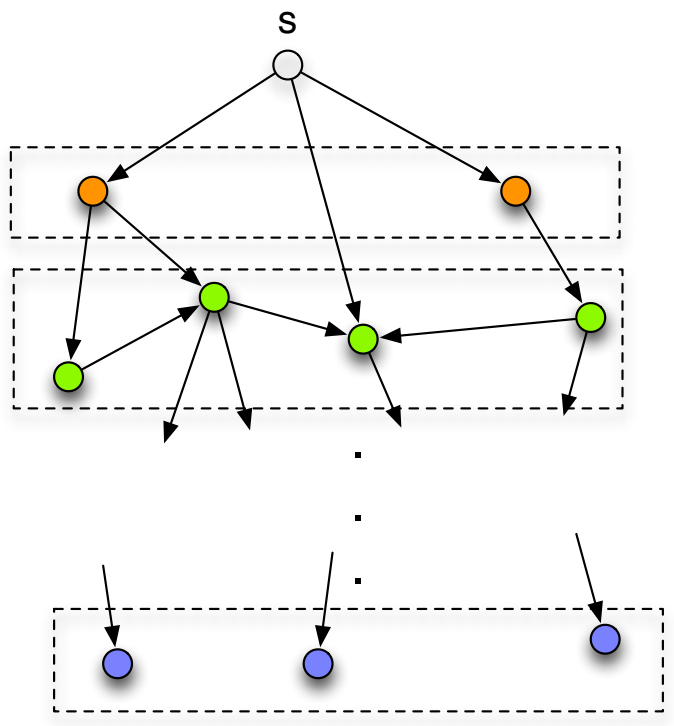


Run Dijkstra algorithm in which d’ and **L(w(u,v))** are intervals

For every outgoing edge (u,v)

d’(v)← max{d’(v), min{d’(u), L(w(u,v))}}

L(w(u,v)): interval for w(u,v)



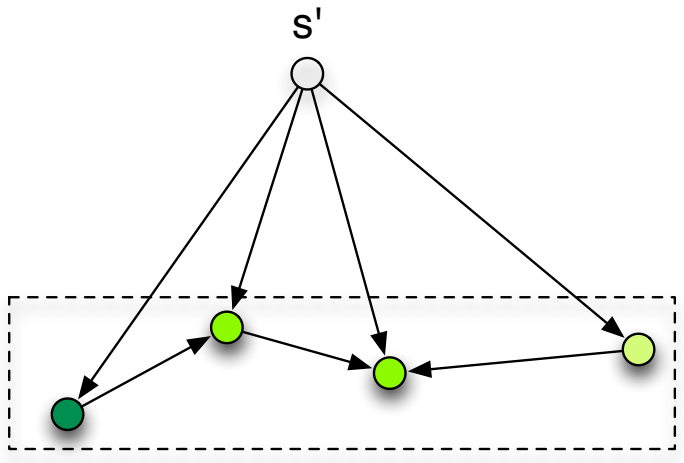
It can partition the graph into “levels”, depending on the bottleneck path is in which interval



Run Dijkstra algorithm:

For every outgoing edge (u,v)

d’(v)← max{d’(v), min{d’(u), L(w(u,v))}}



Then recursively do that for vertices in the same interval

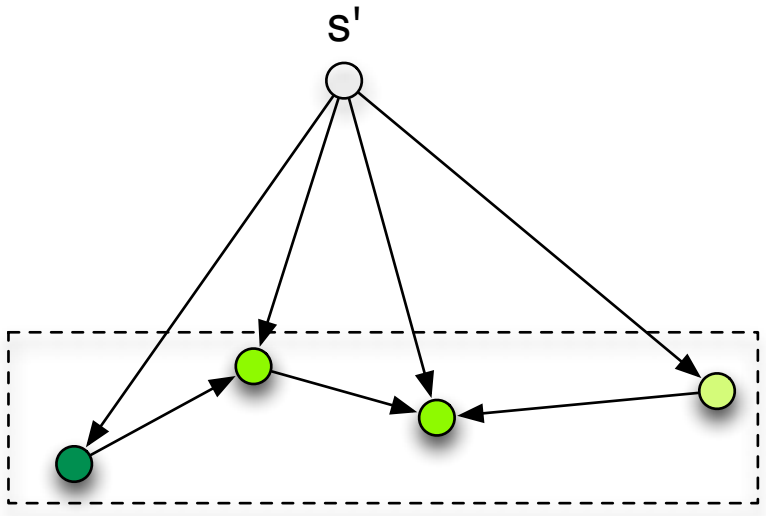
By contracting larger vertices into one node and deleting smaller vertices.



Run Dijkstra algorithm:

For every outgoing edge (u,v)

d'(v)← max{d'(v), min{d'(u), L(w(u,v))}}



Locating w(u,v) into k+1 intervals takes O(log k) time
Each interval has O(n/k) edges on expectation
so #recursion levels is $O(\log_k n)$
In total it is still O(m log n)

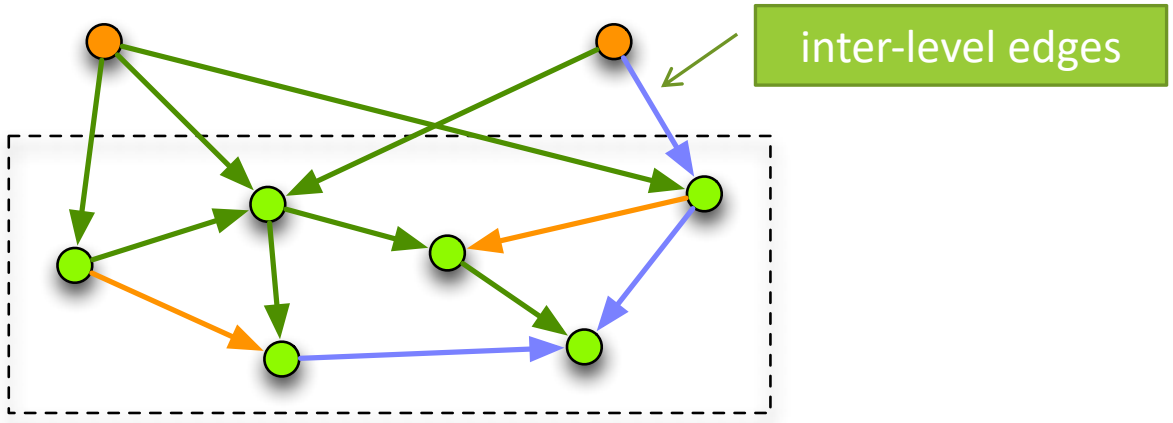
How can we do better?



Detail analysis

To the next recursion level, we consider vertices with d' in one interval:

- For edges from larger intervals, we only keep the largest edge for each vertex

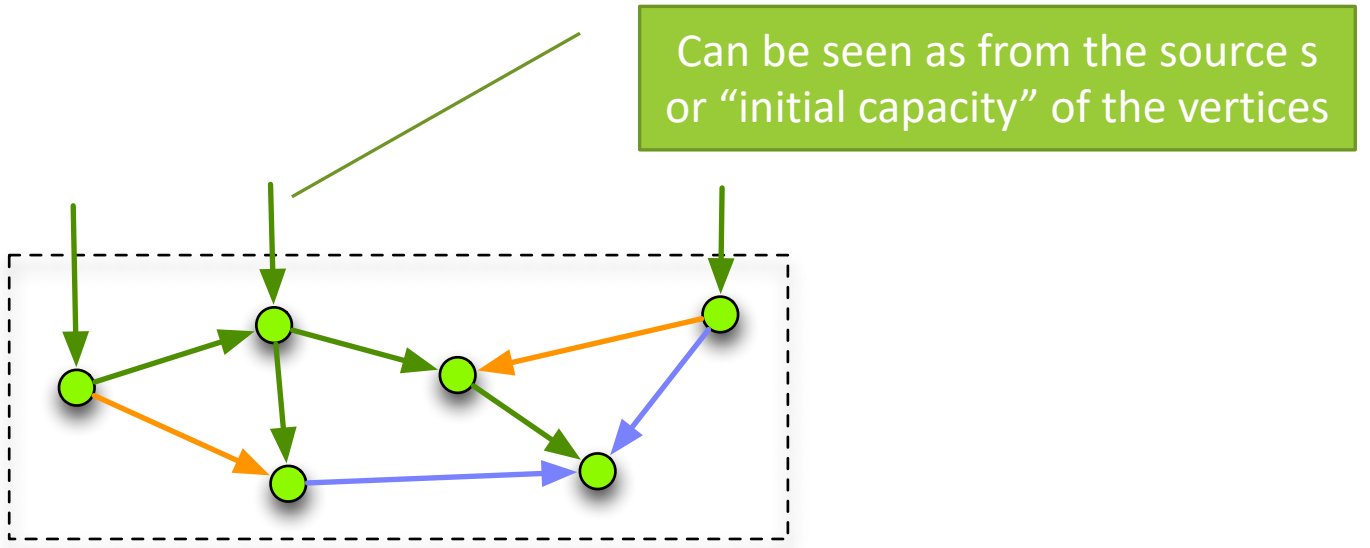




Detail analysis

To the next recursion level, we consider vertices with d’ in one interval:

- For edges from larger intervals, we only keep the largest edge for each vertex

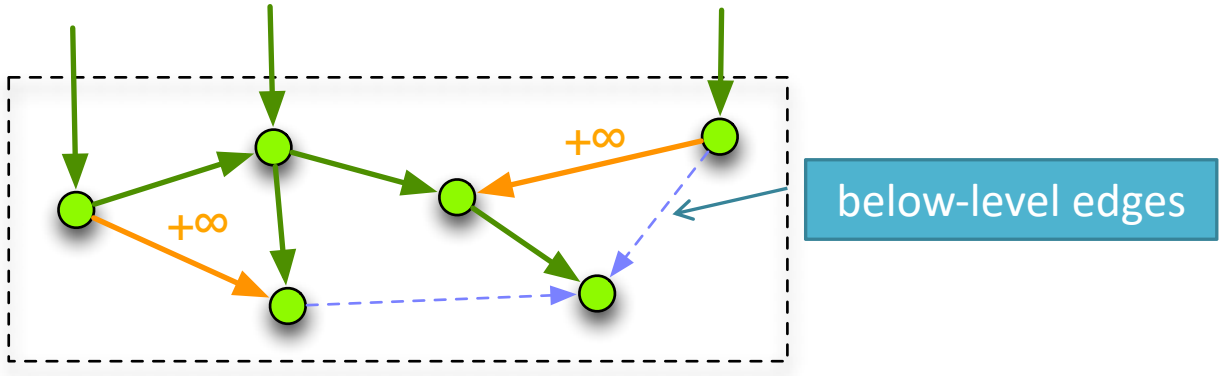




Detail analysis

To the next recursion level, we consider vertices with d’ in one interval:

- For edges between vertices of the same interval, edges with larger intervals are given infinity capacity, and edges with smaller intervals are removed.





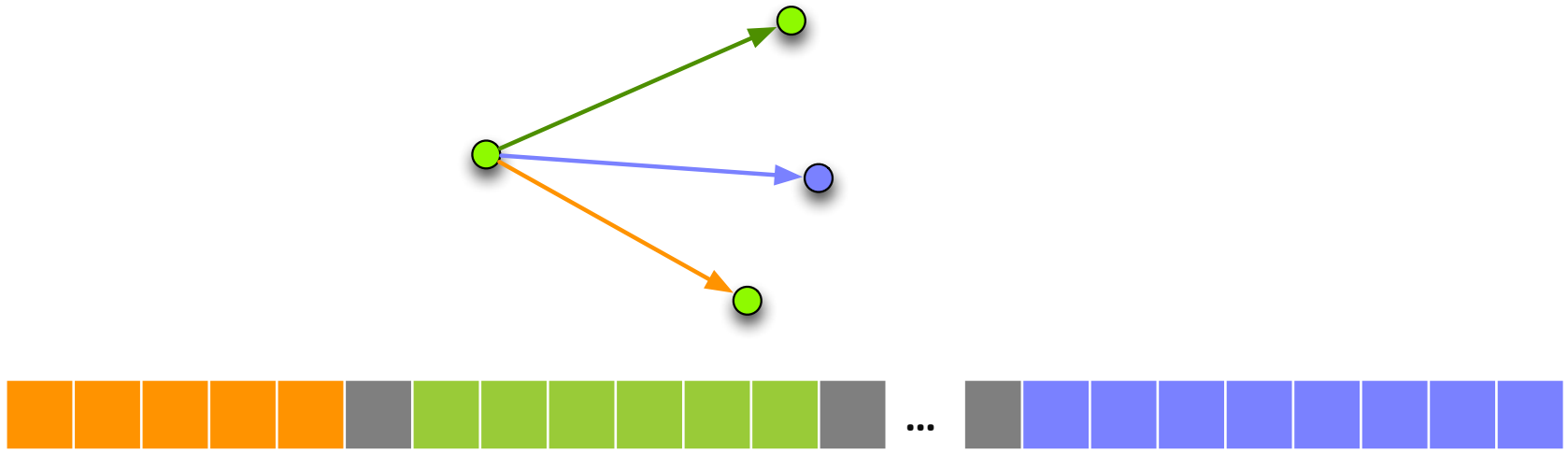
For every outgoing edge (u,v)
d'(v)← max{d'(v), min{d'(u), L(w(u,v))}}

L(w(u,v)): interval for w(u,v)

During Dijkstra…

When we scan edges from u, we don't immediately evaluate L(w(u,v))

- Checking whether L(w(u,v))==d'(u) or d'(v) takes O(1) time!



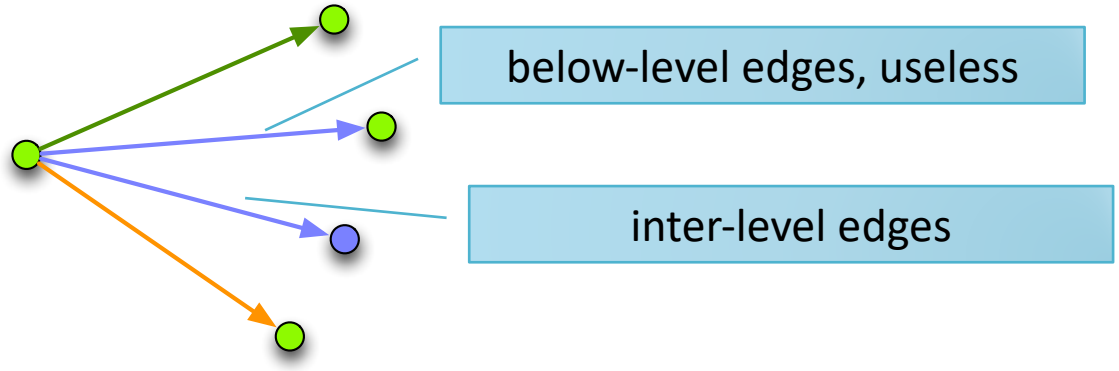
For every outgoing edge (u,v)
d'(v)← max{d'(v), min{d'(u), L(w(u,v))}}

L(w(u,v)): interval for w(u,v)

During Dijkstra…

When we scan edges from u, we don't immediately evaluate L(w(u,v))

- Checking whether L(w(u,v))==d'(u) or d'(v) takes O(1) time!
- Only case when we need to evaluate L(w(u,v)): when d'(u)>L(w(u,v))>d'(v)
- *meaning w(u,v) are in lower interval than d'(u),* then (u,v) becomes "inter-level" edges or "below-level" edges (in the next recursion)





For every outgoing edge (u,v)
d'(v)← max{d'(v), min{d'(u), L(w(u,v))}}

L(w(u,v)): interval for w(u,v)

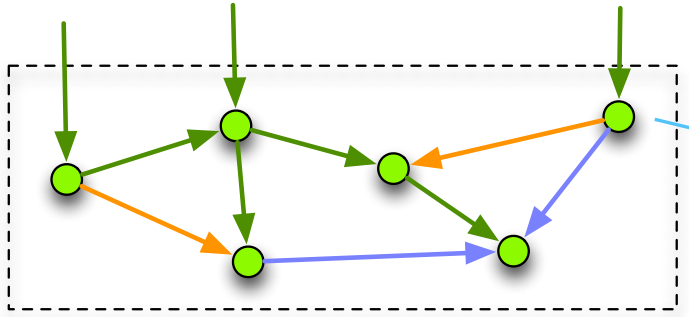
During Dijkstra…

When we scan edges from u, we don't immediately evaluate L(w(u,v))

- Checking whether L(w(u,v))==d'(u) or d'(v) takes O(1) time!
- Only case when we need to evaluate L(w(u,v)): when d'(u)>L(w(u,v))>d'(v)
- *meaning w(u,v) are in lower interval than d'(u),* then (u,v) becomes "inter-level" edges or "below-level" edges (in the next recursion)

Every time we evaluate L(w(u,v)), (u,v) will not be in following recursions

- thus O(log k) time per edge

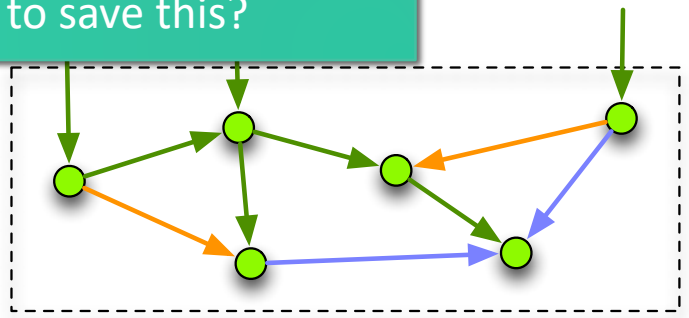


"initial capacites" c(v) for vertex v

Dijkstra with initial capacities

```
d(v) ← L(c(v)), initial capacity for all v
Mark all vertices active
While there is an active vertex
    Select an active u with maximum d(u)
    mark u complete
    Relax every outgoing edge (u,v):
        d(v)← max{d(v), min{d(u),L(w(u,v))}}
```

How to save this?



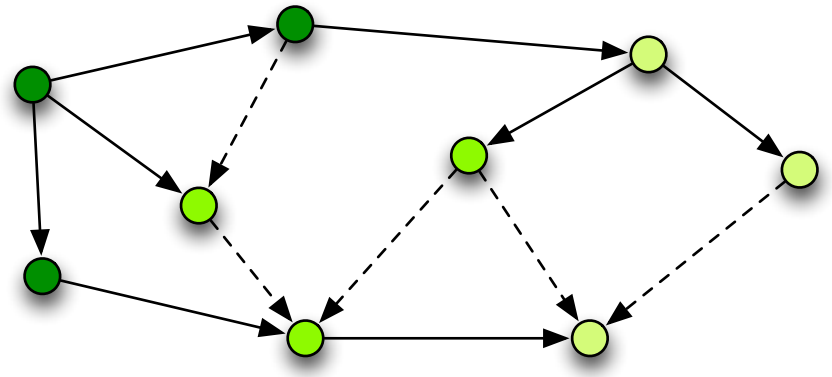
O(log k) time per edge

d’(v) from 1 to k+1, so bucket sort is enough

Subtrees approach

We first find a spanning tree of the graph (treat each edge undirected)

Then partition the spanning tree into subtrees of size O(log n)



Subtrees approach

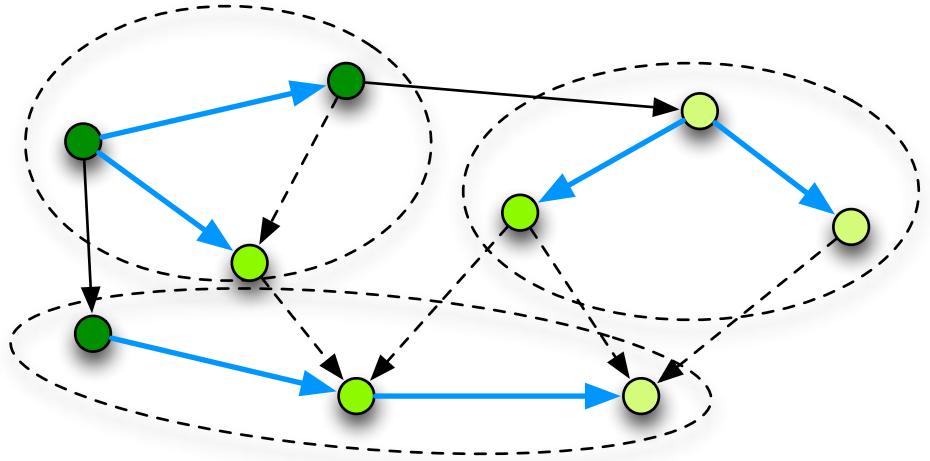
We first find a spanning tree of the graph (treat each edge undirected)

Then partition the spanning tree into subtrees of size O(log n)

Sort the initial capacities for vertices in each subtree

- only takes O(n·loglog n) time

Evaluate L(c(v)) for the largest c(v) of every subtree



Subtrees approach

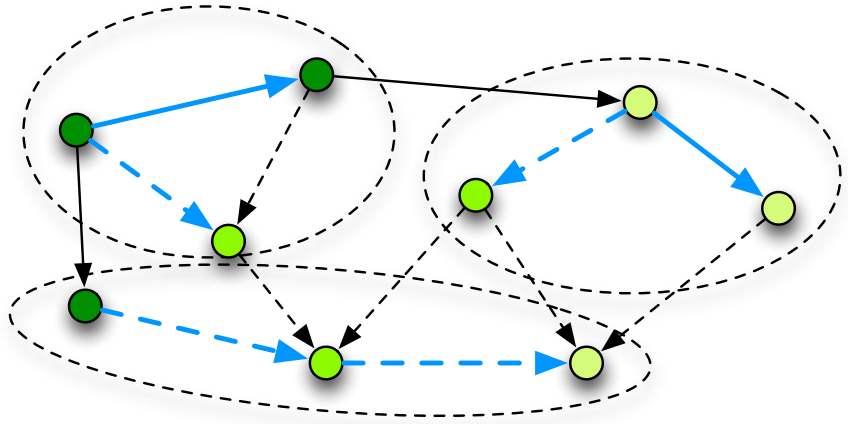
Partition the spanning tree into subtrees of size O(log n)

Sort the initial capacities for vertices in each subtree

Evaluate L(c(v)) for the largest c(v) of every subtree

- Also when we need to evaluate L(c(v)), then the vertices in the subtree will be in different levels
- the edges between different levels will not be in the next recursion!

If we evaluate s initial capacities in this subtree, then s-1 edges of this subtree will be inter-level edges in the next recursion!



Also O(log k) time per edge

Summary

Set $k = 2^{\sqrt{\log n}}$

- Every edge (u,v) only need to be evaluated L(w(u,v)) O(1) times, which takes $O\left(\sqrt{\log n}\right)$ time
- the depth of recursion is $\log_k n = \sqrt{\log n}$
- Other operations takes $O\left(m\sqrt{\log n}\right)$ time

Total time: $O\left(m\sqrt{\log n}\right)$ time

- (we don't need to sort all initial capacities in a subtree, just choose size of subtree be $\sqrt{\log n}$, then use exhausted search)

Variants of Dijkstra's algorithm of O(m+n·log n) time also works for other problems:

- Minimum spanning tree *(Prim-Jarník-Dijkstra),* Bottleneck path, Nondecreasing path

Improvements:

Minimum spanning tree:

- O(m·loglog n) *[Yao 1974]*
- O(mβ(m,n)) *[Fredman & Tarjan 1987]*
- Current best bound:
  - deterministic O(m·α(m,n)) *[Chazelle 1999]*
  - randomized O(m) *[Karger, Klein & Tarjan 1995]*
  - proved optimal *[Pettie and Ramachandran 2002]*

s-t bottleneck path:

- deterministic O(m·log*n) *[Gabow & Tarjan 1988]*
- randomized O(m·β(m,n)) *[Chechik et al. 2016]*

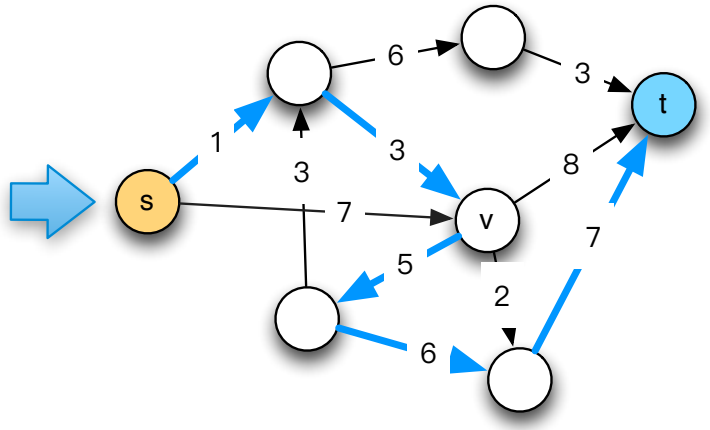
Single-source bottleneck path:

- $O(m\sqrt{\log n})$ *[Duan, Lyu, Wu & Xie 2018]*

Single-source nondecreasing path:

- O(m·loglog n) *[Virginia V.Williams 2009]*

$\beta(m,n)=\min\{i: \log^{(i)} n \le m/n\}$
$\alpha(m,n)$: inverse Ackermann



Nondecreasing path:
nondecreasing edge weights
(minimizing the last one)

Single-source shortest path

Real-weights (comparison-addition model)

- $O(m \cdot \alpha(m,n)+\min\{n \cdot \log n, n \cdot \log\log r\})$ for undirected graph (r is ratio of max-to-min weight)

  [Pettie & Ramachandran 2005]

- Dijkstra's algorithm is universally optimal for *distance ordering* problem

  [Haeupler, Hladík, Rozhoň, Tarjan, Tětek 2024]

Undirected SSSP: randomized $O\left(m\sqrt{\log n \cdot \log\log n}\right)$ time *[Duan, Mao, Shu, Yin, 2023]*

- made deterministic *[Yan, SOSA 2026]*

Directed SSSP: deterministic $O\left(m \log^{2/3} n\right)$ time *[Duan, Mao, Mao, Shu, Yin, 2025]*

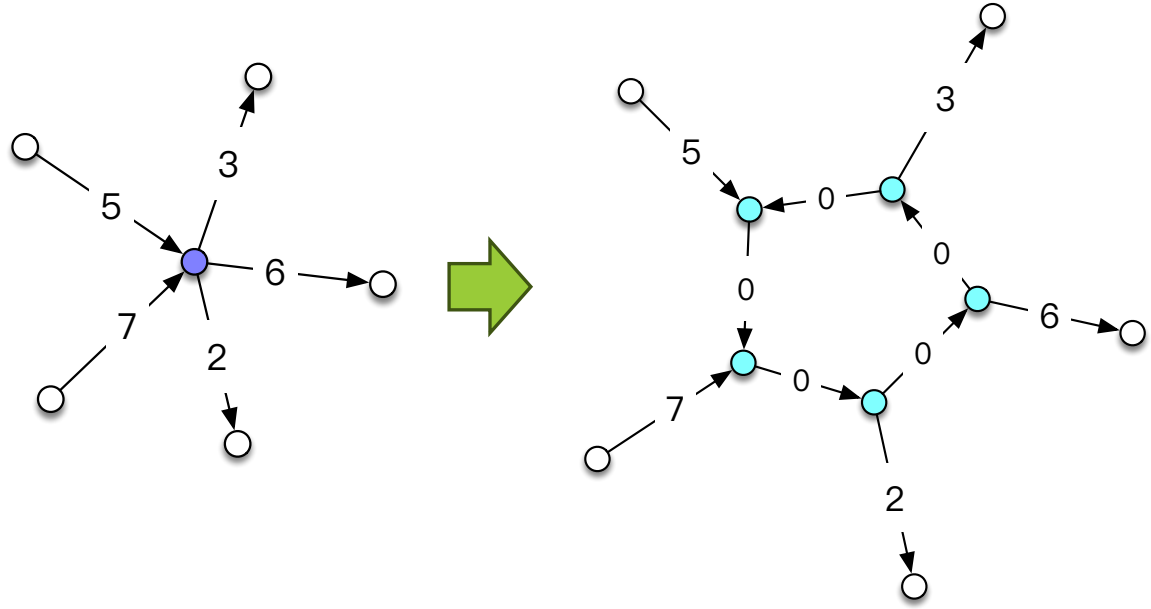
- improved to deterministic $O\left(m\sqrt{\log n}\log\log n\right)$ time *[Duan, Mao, Shu, Yin, 2026]*

Our algorithm

Directed SSSP: $O\left(m \log^{2/3} n\right)$ time

Make the graph sparse enough

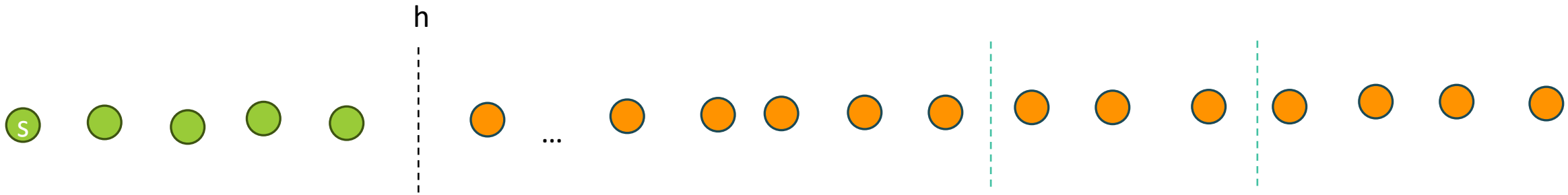
- Every vertex of constant degree
- m=O(n)



At some stage of Dijkstra's, all vertices u with dist(s,u)<h are complete
Heap needs to contain all vertices relaxed by complete vertices

To save sorting time, what about "partially" sorting them?

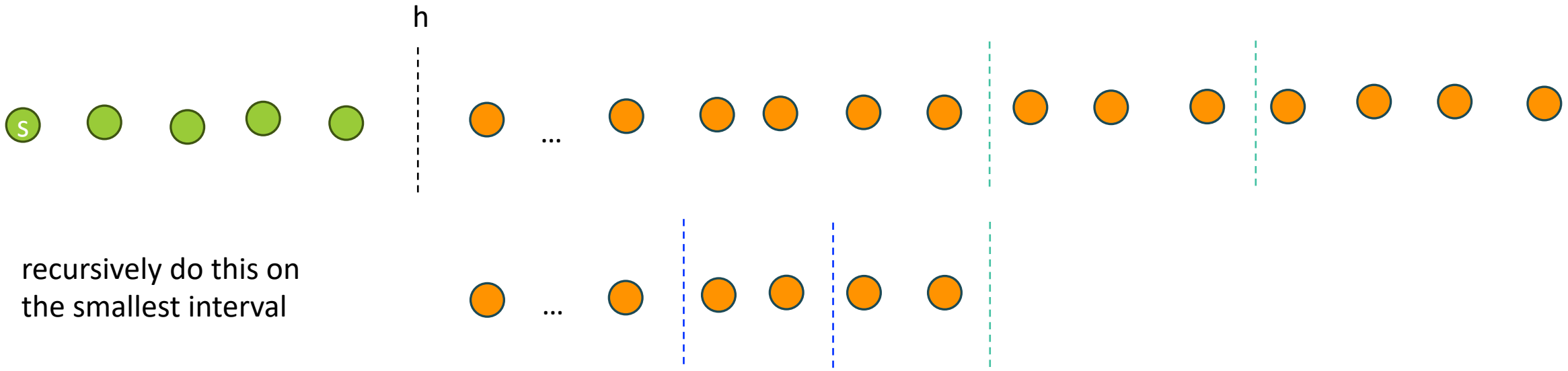
partition into 2^t intervals by order
O(t) time per edge



At some stage of Dijkstra's, all vertices u with dist(s,u)<h are complete
Heap needs to contain all vertices relaxed by complete vertices

To save sorting time, what about "partially" sorting them?

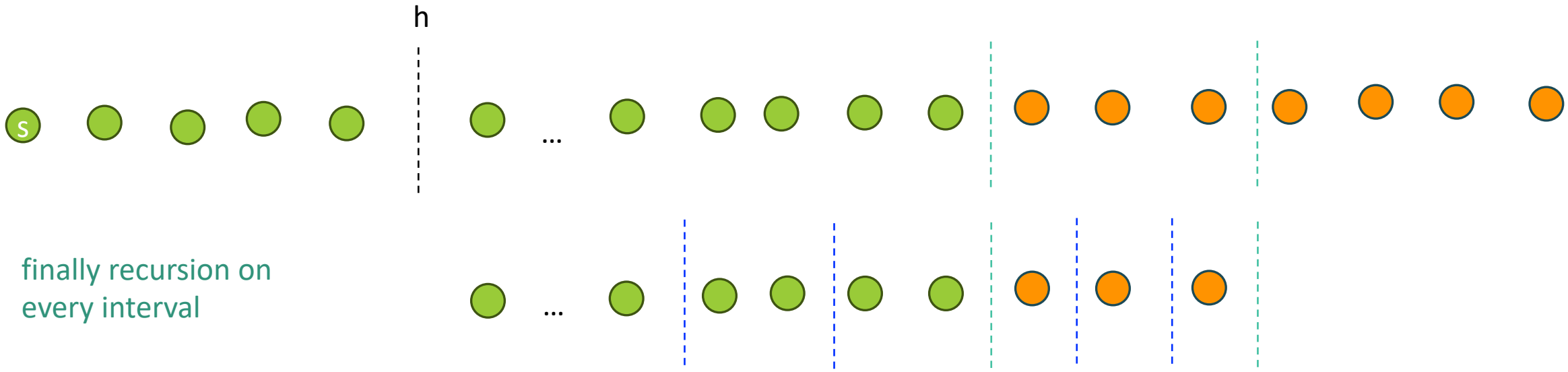
partition into 2^t intervals by order
O(t) time per edge



At some stage of Dijkstra's, all vertices u with dist(s,u)<h are complete
Heap needs to contain all vertices relaxed by complete vertices

To save sorting time, what about "partially" sorting them?

partition into 2^t intervals by order
O(t) time per edge

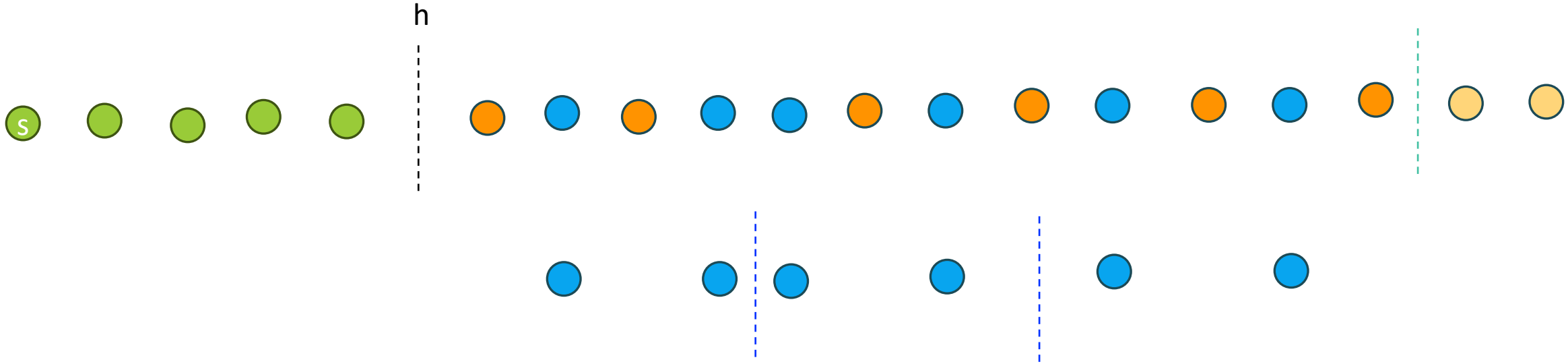


We need O((log n)/t) levels to fully sort vertices, also O(log n) time per edge

At some stage of Dijkstra's, all vertices u with dist(s,u)<h are complete
Heap needs to contain all vertices relaxed by complete vertices

SOLUTION: reduce the number of vertices partially sorted in the next level

partition into 2^t intervals by order
O(t) time per edge



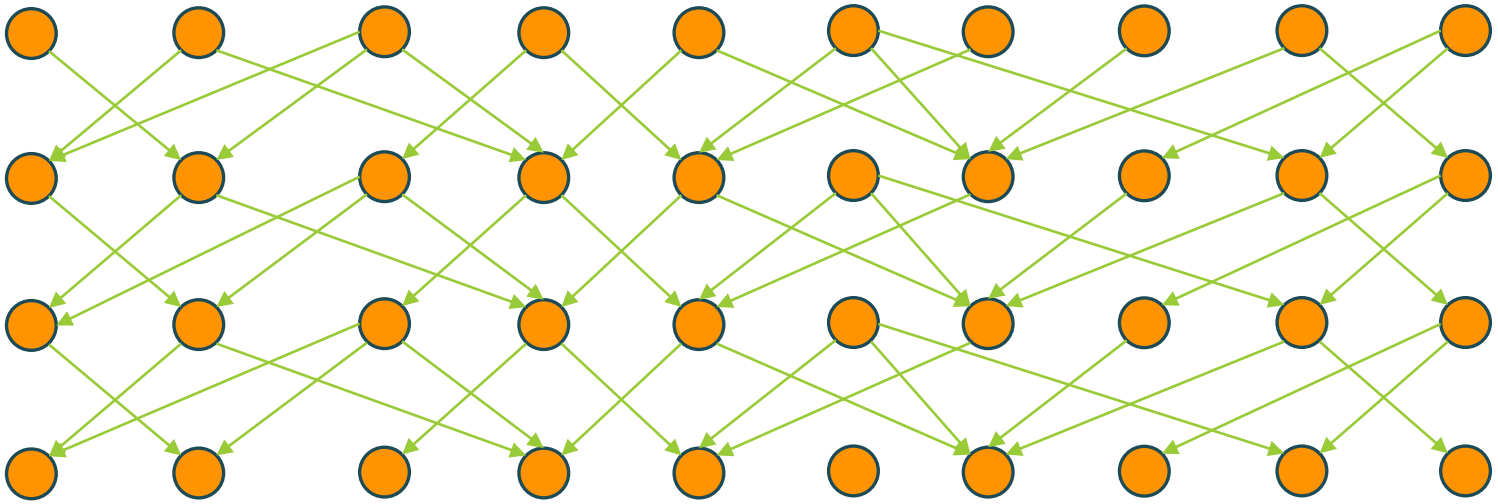
Bellman–Ford algorithm

Direct dynamic programming, no need to sort

```
Repeat n-1 times:
    Relax every edge (u,v):
        d(v)← min{d(v), d(u)+w(u,v)}
```

After k iterations, we can find the shortest path with ≤ k edges

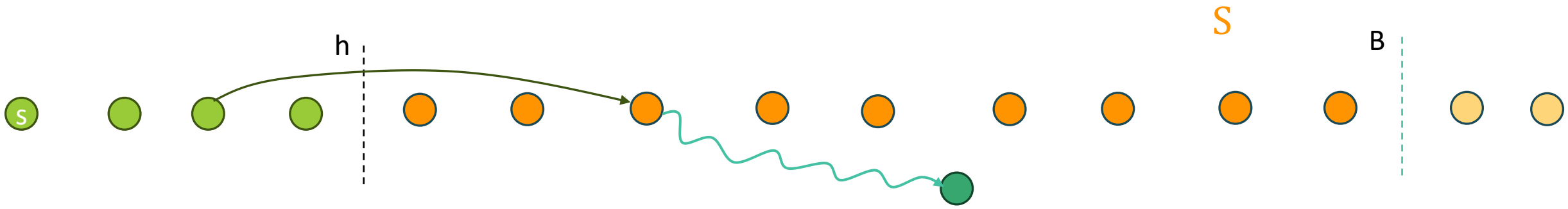
Works also for graphs with negative edge weights



Right now all vertices u with dist(s,u)<h are complete, and we have relaxed their edges

Consider vertices v with current h≤d(v)<B, call the set S

- ✓ for any w with h≤dist(s,w)<B, the shortest path to w must visit some vertex in S



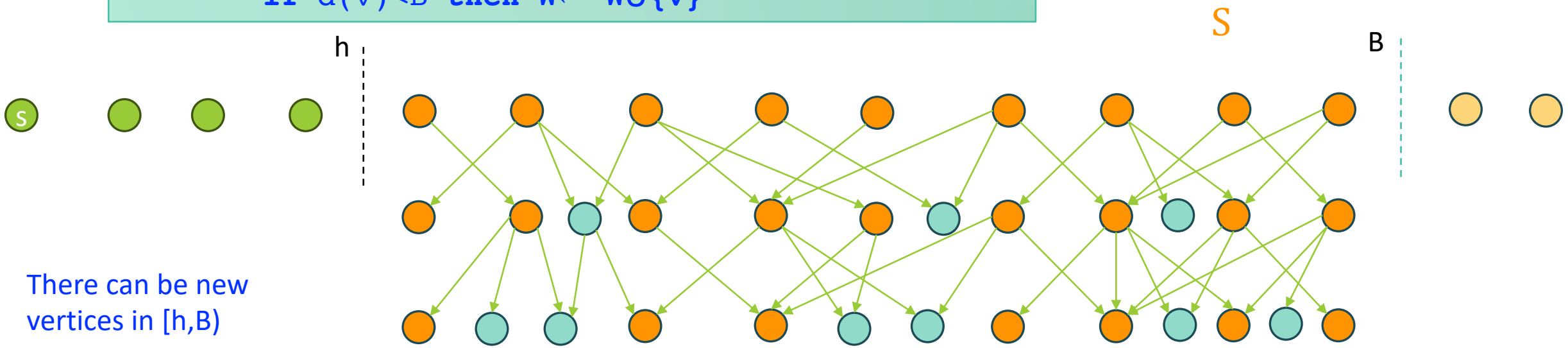
Right now all vertices u with dist(s,u)<h are complete, and we have relaxed their edges

Consider vertices v with current h≤d(v)<B, call the set S

✓ for any w with h≤dist(s,w)<B, the shortest path to w must visit some vertex in S

Do Bellman-Ford from S for k steps:

```
W← S
Repeat k times or until |W|>k|S|:
    Relax every edge (u,v) from u∈W:
        d(v)← min{d(v), d(u)+w(u,v)}
        if d(v)<B then W← W∪{v}
```



There can be new vertices in [h,B)

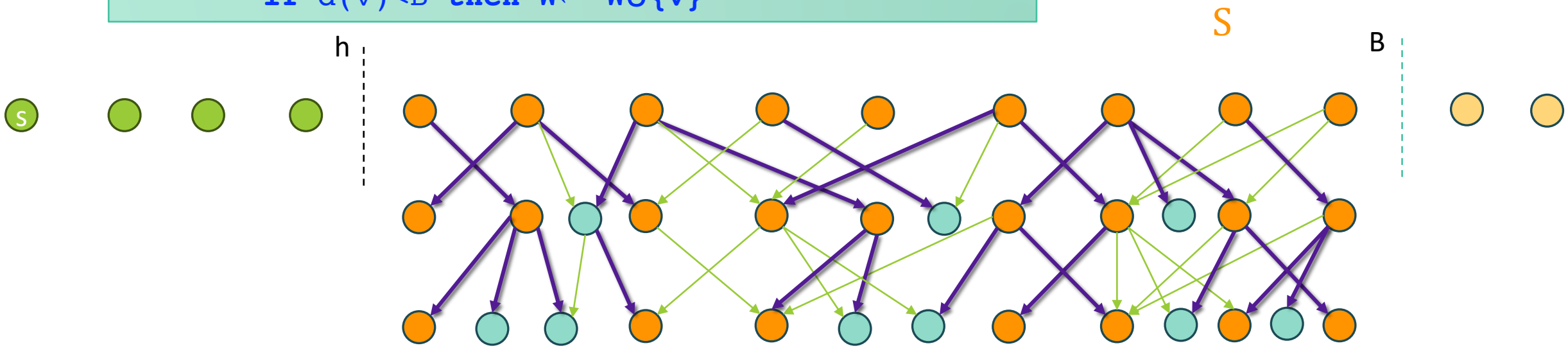
Right now all vertices u with dist(s,u)<h are complete, and we have relaxed their edges

Consider vertices v with current h≤d(v)<B, call the set S

✓ for any w with h≤dist(s,w)<B, the shortest path to w must visit some vertex in S

Do Bellman-Ford from S for k steps:

```
W← S
Repeat k times or until |W|>k|S|:
    Relax every edge (u,v) from u∈W:
        d(v)← min{d(v), d(u)+w(u,v)}
        if d(v)<B then W← W∪{v}
```



Find current shortest path trees rooted at S

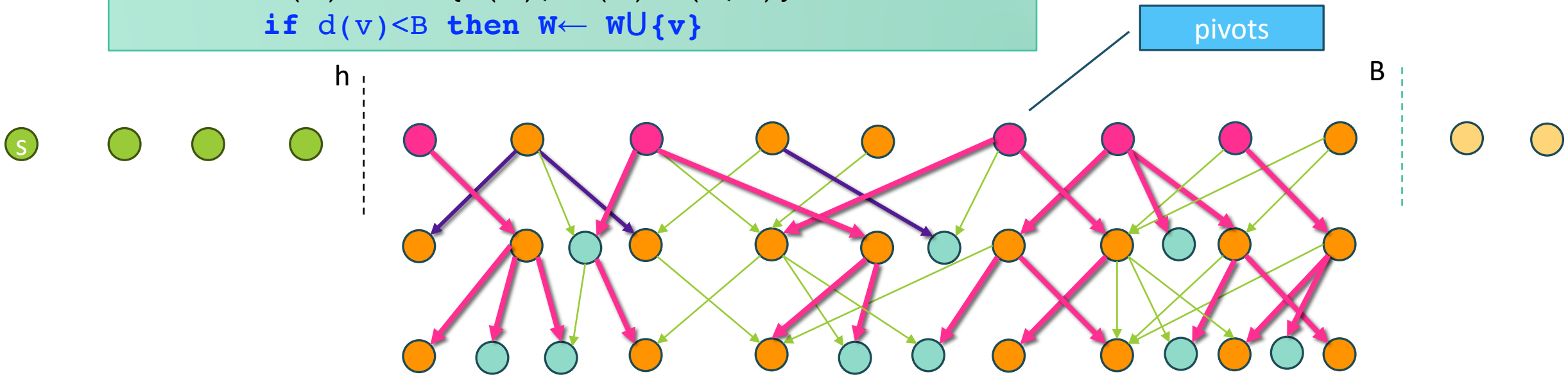
Right now all vertices u with dist(s,u)<h are complete, and we have relaxed their edges

Consider vertices v with current h≤d(v)<B, call the set S

✓ for any w with h≤dist(s,w)<B, the shortest path to w must visit some vertex in S

Do Bellman-Ford from S for k steps:

```
W← S
Repeat k times or until |W|>k|S|:
    Relax every edge (u,v) from u∈W:
        d(v)← min{d(v), d(u)+w(u,v)}
        if d(v)<B then W← W∪{v}
```



trees reaching last level are still growing, their roots are called “pivots”

if there are k B-F steps, each tree with size >k

Right now all vertices u with dist(s,u)<h are complete

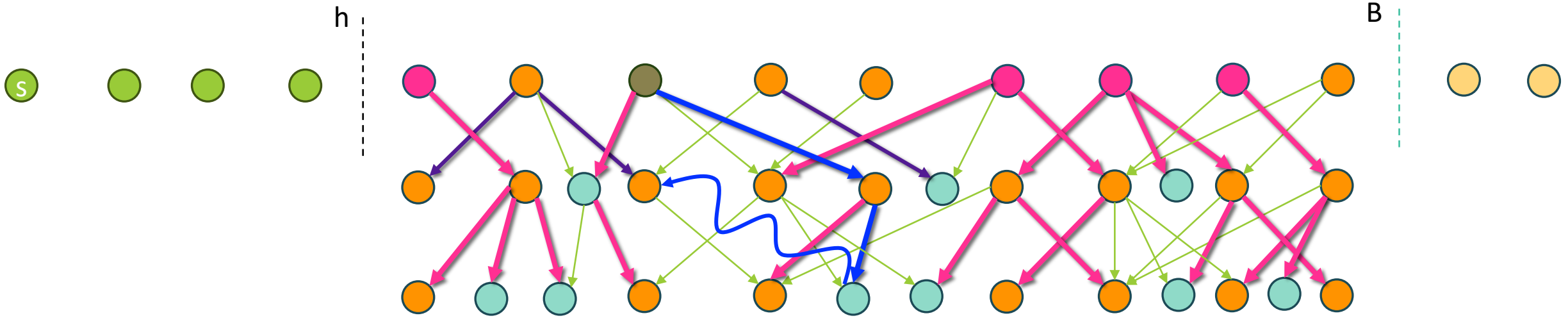
Consider vertices v with current h≤d(v)<B, call the set S

✓ for any w with h≤dist(s,w)<B, the shortest path to w must visit some vertex in S

Do Bellman-Ford from S for k steps:

```
W← S
Repeat k times or until |W|>k|S|:
    Relax every edge (u,v) from u∈W:
        d(v)← min{d(v), d(u)+w(u,v)}
        if d(v)<B then W← W∪{v}
```

if w with dist(s,w)<B is still incomplete, then its shortest path must go through such a pivot



trees reaching last level are still growing, their roots are called “pivots”

if there are k B-F steps, each tree with size >k

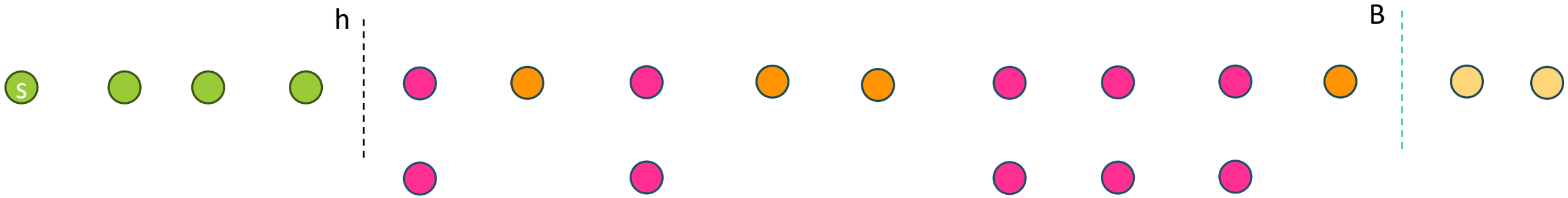
Right now all vertices u with dist(s,u)<h are complete, and we have relaxed their edges

Consider vertices v with current h≤d(v)<B, call the set S

✓ for any w with h≤dist(s,w)<B, the shortest path to w must visit some vertex in S

```
W← S
Repeat k times or until |W|>k|S|:
    Relax every edge (u,v) from u∈W:
        d(v)← min{d(v), d(u)+w(u,v)}
        if d(v)<B then W← W∪{v}
if |W|≤k|S| then P← set of pivots else P← S
Return P,W
```

$\tilde{U}$={w| h≤dist(s,w)<B and its shortest path goes through S}

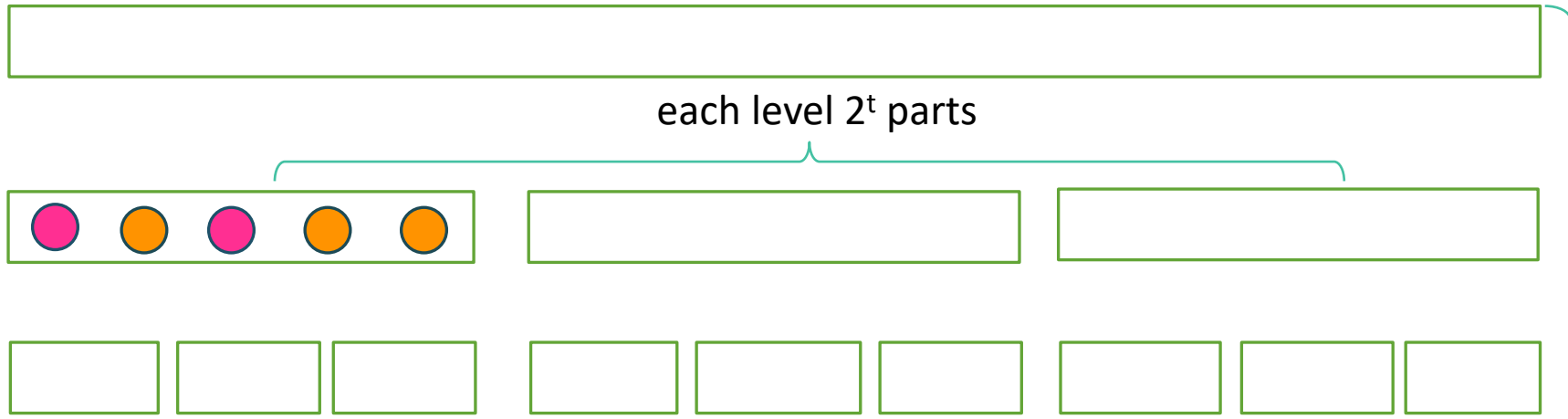


- In $O(min\{k^2|S|, k|\tilde{U}|\})$ time, we get $|\tilde{U}|/k$ pivots of S which need to be sorted in next level
- Shortest paths to $\tilde{U}$ not going through these pivots are already complete

Ideal case: $|\tilde{U}|=O(|S|)$

Rough Idea

Recursively partitioning:



In each level, O(n/k) vertices partially sorted in next level

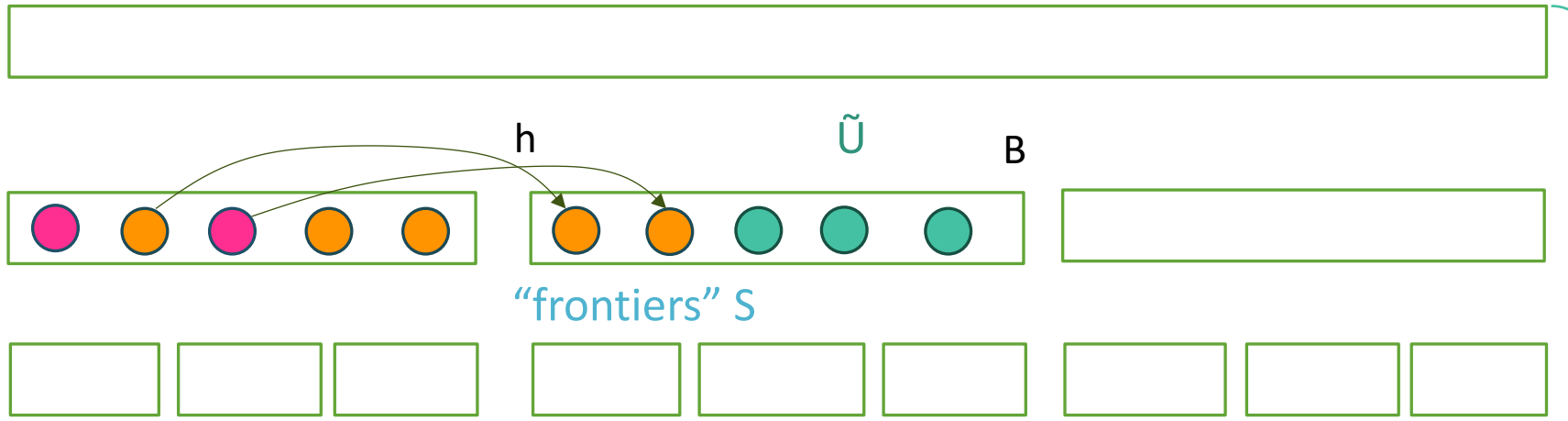
- Partial sorting takes $O\left(mt + \frac{n}{k} \cdot t \cdot \frac{\log n}{t}\right) = O\left(mt + \frac{n \log n}{k}\right)$
- Bellman-Ford steps take $O\left(nk \cdot \frac{\log n}{t}\right)$

Let $t = \log^{2/3} n$, $k = \log^{1/3} n$, total $O\left(m \log^{2/3} n\right)$

Rough Idea

- Ũ may be far larger than S
- How to choose bound B to make recursive partition balanced?
- *SOLUTION: if one part getting too large, stop and change B*

Recursively partitioning:



In each level, O(n/k) vertices partially sorted in next level

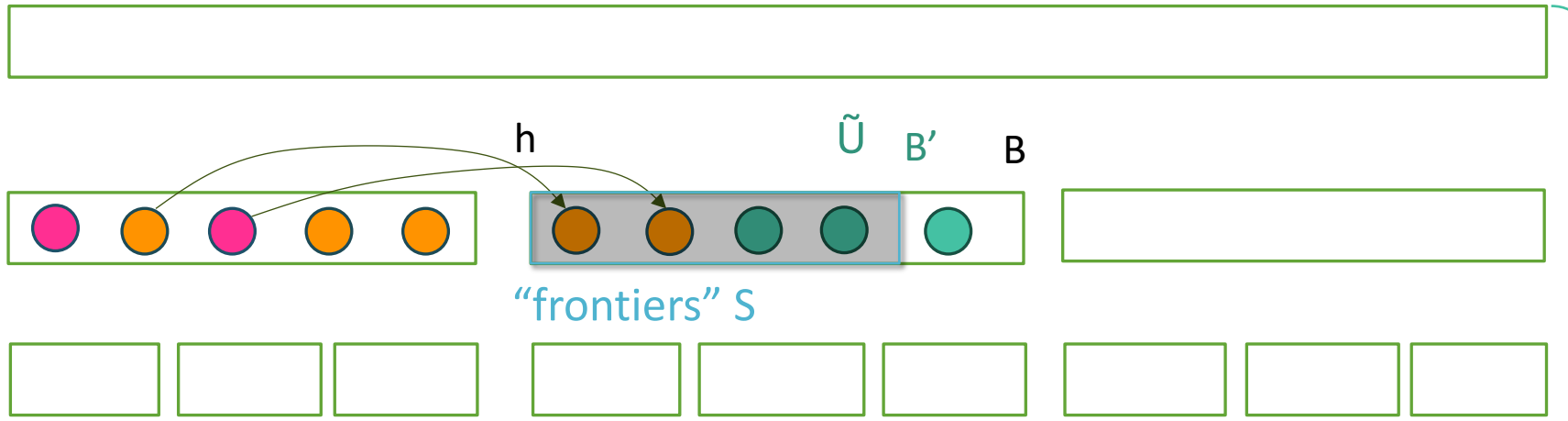
- Partial sorting takes $O\left(mt + \frac{n}{k} \cdot t \cdot \frac{\log n}{t}\right) = O\left(mt + \frac{n \log n}{k}\right)$
- Bellman-Ford steps take $O\left(nk \cdot \frac{\log n}{t}\right)$

Let $t = \log^{2/3} n$, $k = \log^{1/3} n$, total $O\left(m \log^{2/3} n\right)$

Rough Idea

- Ũ may be far larger than S
- How to choose bound B to make recursive partition balanced?
- *SOLUTION: if one part getting too large, stop and change B*
  - *Successful execution: B'=B*
  - *Partial execution: B'<B*

Recursively partitioning:



In each level, O(n/k) vertices partially sorted in next level

- Partial sorting takes $O\left(mt + \frac{n}{k} \cdot t \cdot \frac{\log n}{t}\right) = O\left(mt + \frac{n \log n}{k}\right)$
- Bellman-Ford steps take $O\left(nk \cdot \frac{\log n}{t}\right)$

Let $t = \log^{2/3} n$, $k = \log^{1/3} n$, total $O\left(m \log^{2/3} n\right)$

Main algorithm: Bounded Multi-Source Shortest Path

```
function BMSSP(ℓ,B,S)
    BASE CASE: if ℓ=0 then use Dijkstra's algorithm
    P,W ← Bellman-Ford(B,S)
    Initialize 𝒟 to contain P
    i←0; U←∅
    while |U|<k2^(ℓt) and 𝒟 non-empty
        i++
        S_i ← pull 2^((ℓ-1)t) elements from 𝒟, with bound B_i
        (B_i',U_i)←BMSSP(ℓ-1,B_i,S_i)
        U←U ∪ U_i
        K←{v|v∈S_i,d(v)∈[B_i',B_i)}
        for all edge (u,v) where u∈U_i
            if d(u)+w(u,v)≤d(v) then
                d(v)←d(u)+w(u,v)
                if d(u)+w(u,v)∈[B_i,B) then
                    insert v into 𝒟
                elseif d(u)+w(u,v)∈[B_i',B_i) then
                    insert v into K
        batch-prepend K to 𝒟
    B'←min{B_i',B}; U←U ∪ {x∈W | d(x)<B'}
    return B',U
```

$k = \log^{1/3} n$, $t = \log^{2/3} n$

INPUT: **frontiers S** and $|S| \le 2^{\ell t}$, with bound B

OUTPUT:

- boundary $B' \le B$
- complete set U containing all vertex u with dist(s,u)<B' and its shortest path visits S

TOP LEVEL:

- call **BMSSP(ℓ=(log n)/t,B=+∞,S={s})**

Main algorithm: Bounded Multi-Source Shortest Path

```
function BMSSP(ℓ,B,S)
    BASE CASE: if ℓ=0 then use Dijkstra's algorithm
    P,W ← Bellman-Ford(B,S)
    Initialize 𝒟 to contain P
    i←0; U←∅
    while |U|<k2^(ℓt) and 𝒟 non-empty
        i++
        S_i ← pull 2^((ℓ-1)t) elements from 𝒟, with bound B_i
        (B_i',U_i)←BMSSP(ℓ-1,B_i,S_i)
        U←U ∪ U_i
        K←{v|v∈S_i,d(v)∈[B_i',B_i)}
        for all edge (u,v) where u∈U_i
            if d(u)+w(u,v)≤d(v) then
                d(v)←d(u)+w(u,v)
                if d(u)+w(u,v)∈[B_i,B) then
                    insert v into 𝒟
                elseif d(u)+w(u,v)∈[B_i',B_i) then
                    insert v into K
        batch-prepend K to 𝒟
    B'←min{B_i',B}; U←U ∪ {x∈W | d(x)<B'}
    return B',U
```

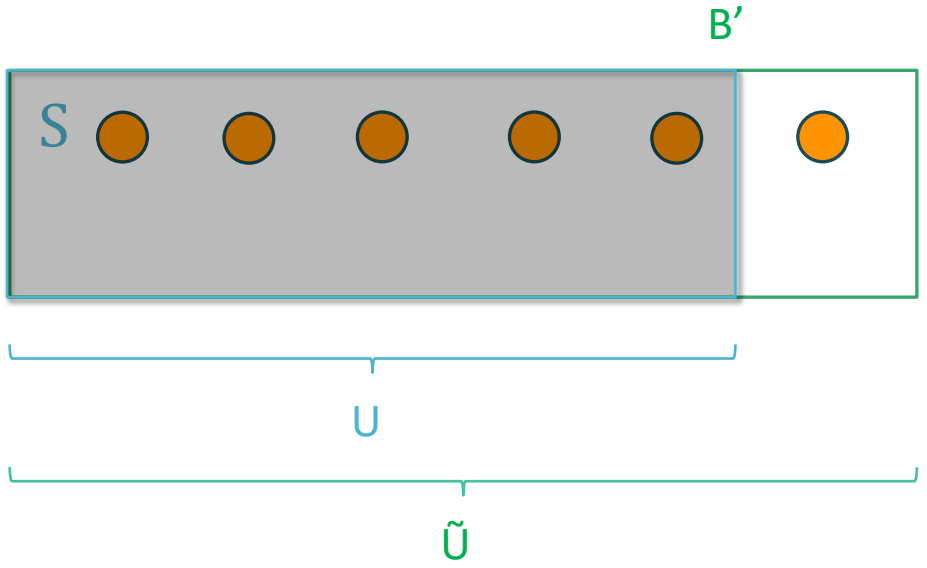
$k = \log^{1/3} n$, $t = \log^{2/3} n$

INPUT: frontiers S and $|S| \le 2^{\ell t}$, with bound B

OUTPUT:

- boundary $B' \le B$
- complete set U containing all vertex u with dist(s,u)<B' and its shortest path visits S

Successful execution: B'=B
Partial execution: B'<B, so $|U| = \Theta(k2^{\ell t})$



Data structure $\mathcal{D}$

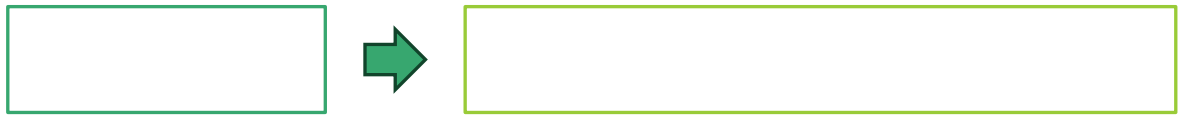
N elements in total, a parameter M

Operations:

- **INSERT** Insert an element, *in O(log(N/M)) amortized time*
- **BATCH-PREPEND** Insert L elements which is smaller than any current value, *in O(L·log(L/M)) time*
- **PULL** Extract M smallest elements, *in O(M) amortized time*

Here $N=O(k2^{\ell t})$, $M=2^{(\ell-1)t}$, $L=O(k2^{(\ell-1)t})$

- So insert takes O(t+log k)=O(t) time, batch-prepend takes O(log k) time per element



Main algorithm: Bounded Multi-Source Shortest Path

```
function BMSSP(ℓ,B,S)
    BASE CASE: if ℓ=0 then use Dijkstra's algorithm
    P,W ← Bellman-Ford(B,S)
    Initialize 𝒟 to c
    i←0; U←∅
    while |U|<k2^(ℓt) and
        i++
        S_i ← pull 2^((ℓ-1)t) elements from 𝒟, with bound B_i
        (B_i',U_i)←BMSSP(ℓ-1,B_i,S_i)
        U←U ∪ U_i
        K←{v|v∈S_i,d(v)∈[B_i',B_i)}
        for all edge (u,v) where u∈U_i
            if d(u)+w(u,v)≤d(v) then
                d(v)←d(u)+w(u,v)
                if d(u)+w(u,v)∈[B_i,B) then
                    insert v into 𝒟
                elseif d(u)+w(u,v)∈[B_i',B_i) then
                    insert v into K
        batch-prepend K to 𝒟
    B'←min{B_i',B}; U←U ∪ {x∈W | d(x)<B'}
    return B',U
```

Here is "≤" since each edge needs to be relaxed in multiple levels

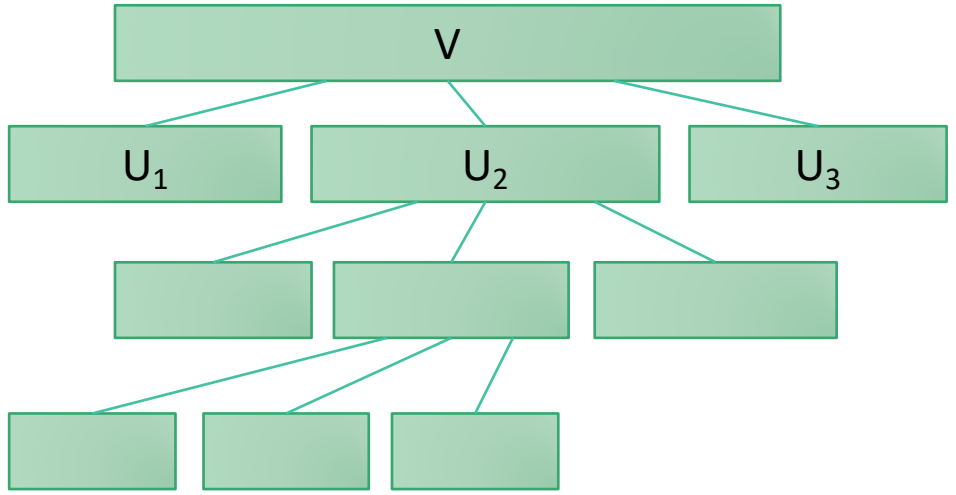
$k = \log^{1/3} n$, $t = \log^{2/3} n$

Recursion tree:

- depth: O((log n)/t), almost balanced
- in each level the sets U are disjoint
- total size of sets U: O(n·(log n)/t)

total time for B-F: O(nk·(log n)/t)

total size for pivots P: O(n·(log n)/(kt)), so inserting them into $\mathcal{D}$ takes O(n·(log n)/k) time



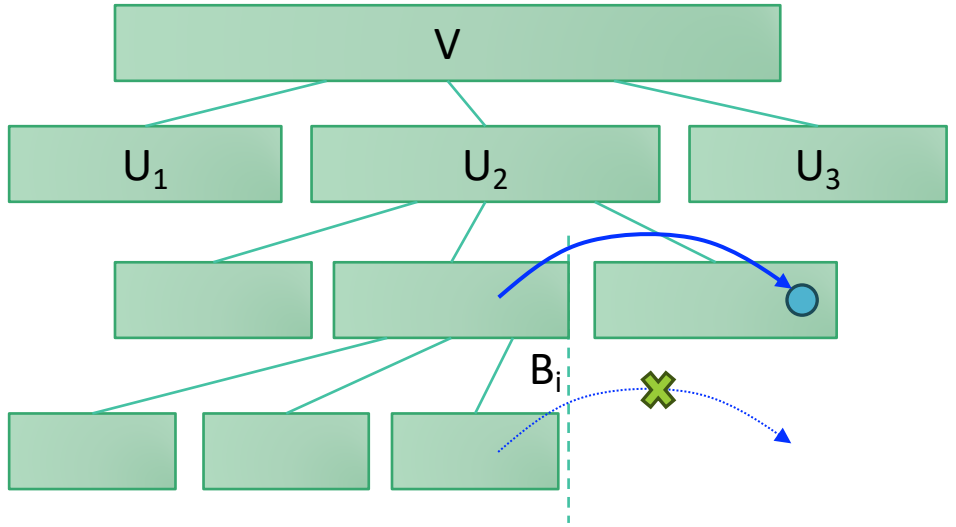
Main algorithm: Bounded Multi-Source Shortest Path

```
function BMSSP(ℓ,B,S)
    BASE CASE: if ℓ=0 then use Dijkstra's algorithm
    P,W ← Bellman-Ford(B,S)
    Initialize 𝒟 to contain P
    i←0; U←∅
    while |U|<k2^(ℓt) and 𝒟 non-empty
        i++
        S_i ← pull 2^((ℓ-1)t) elements from 𝒟, with bound B_i
        (B_i',U_i)←BMSSP(ℓ-1,B_i,S_i)
        U←U ∪ U_i
        K←{v|v∈S_i,d(v)∈[B_i',B_i)}
        for all edge (u,v) where u∈U_i
            if d(u)+w(u,v)≤d(v) then
                d(v)←d(u)+w(u,v)
                if d(u)+w(u,v)∈[B_i,B) then
                    insert v into 𝒟
                elseif d(u)+w(u,v)∈[B_i',B_i) then
                    insert v into K
        batch-prepend K to 𝒟
    B'←min{B_i',B}; U←U ∪ {x∈W | d(x)<B'}
    return B',U
```

If a vertex is directly inserted to $\mathcal{D}$ in multiple levels, total time can still be O(log n)

But if a vertex v is directly inserted to $\mathcal{D}$ by (u,v)

- d(u)+w(u,v) larger than original B_i, and B_i is also a bound for its lower level recursive calls
- so (u,v) will not add v into $\mathcal{D}$ in lower levels



Main algorithm: Bounded Multi-Source Shortest Path

```
function BMSSP(ℓ,B,S)
    BASE CASE: if ℓ=0 then use Dijkstra's algorithm
    P,W ← Bellman-Ford(B,S)
    Initialize 𝒟 to contain P
    i←0; U←∅
    while |U|<k2^(ℓt) and 𝒟 non-empty
        i++
        S_i ← pull 2^((ℓ-1)t) elements from 𝒟, with bound B_i
        (B_i',U_i)←BMSSP(ℓ-1,B_i,S_i)
        U←U ∪ U_i
        K←{v|v∈S_i,d(v)∈[B_i',B_i)}
        for all edge (u,v) where u∈U_i
            if d(u)+w(u,v)≤d(v) then
                d(v)←d(u)+w(u,v)
                if d(u)+w(u,v)∈[B_i,B) then
                    insert v into 𝒟
                elseif d(u)+w(u,v)∈[B_i',B_i) then
                    insert v into K
        batch-prepend K to 𝒟
    B'←min{B_i',B}; U←U ∪ {x∈W | d(x)<B'}
    return B',U
```

If a vertex is directly inserted to $\mathcal{D}$ in multiple levels, total time can still be O(log n)

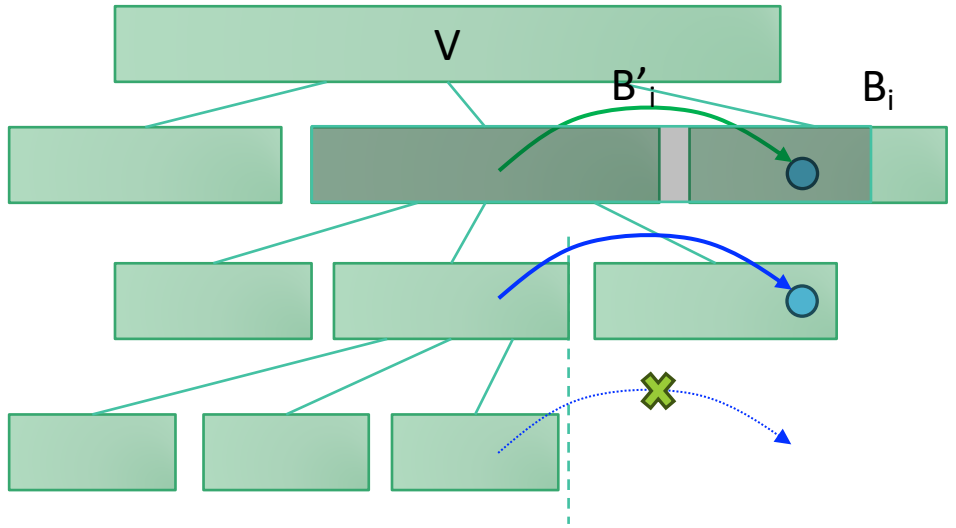
But if a vertex v is directly inserted to $\mathcal{D}$ by (u,v)

- d(u)+w(u,v) larger than original B_i, and B_i is also a bound for its lower level recursive calls
- so (u,v) will not add v into $\mathcal{D}$ in lower levels

Thus v can only be directed inserted to $\mathcal{D}$ in one level by (u,v) in O(t) time

In higher levels it can only be inserted through **K**, in O(log k) time

Total time: O(m(t+(log n·loglog n)/t))=O(m·t)



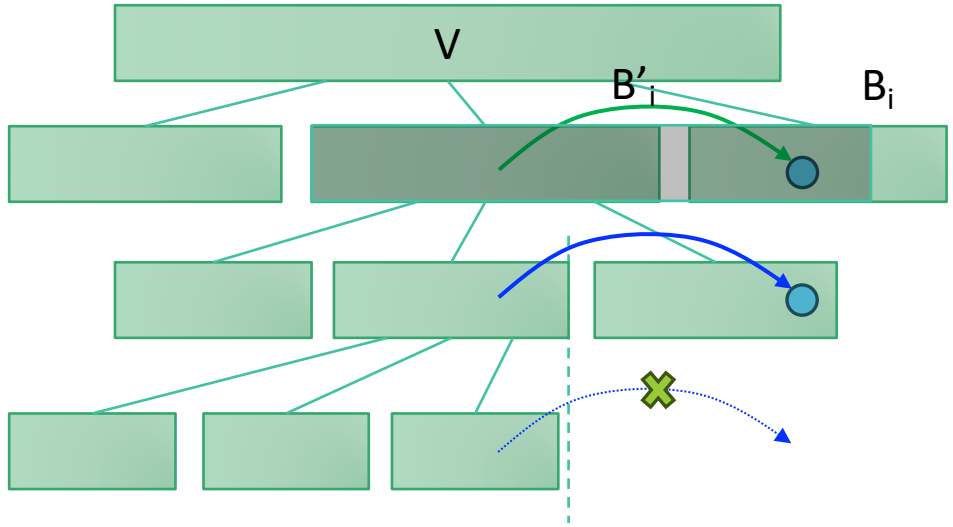
Main algorithm: Bounded Multi-Source Shortest Path

```
function BMSSP(ℓ,B,S)
    BASE CASE: if ℓ=0 then use Dijkstra's algorithm
    P,W ← Bellman-Ford(B,S)
    Initialize 𝒟 to contain P
    i←0; U←∅
    while |U|<k2^(ℓt) and 𝒟 non-empty
        i++
        S_i ← pull 2^((ℓ-1)t) elements from 𝒟, with bound B_i
        (B_i',U_i)←BMSSP(ℓ-1,B_i,S_i)
        U←U ∪ U_i
        K←{v|v∈S_i,d(v)∈[B_i',B_i)}
        for all edge (u,v) where u∈U_i
            if d(u)+w(u,v)≤d(v) then
                d(v)←d(u)+w(u,v)
                if d(u)+w(u,v)∈[B_i,B) then
                    insert v into 𝒟
                elseif d(u)+w(u,v)∈[B_i',B_i) then
                    insert v into K
        batch-prepend K to 𝒟
    B'←min{B_i',B}; U←U ∪ {x∈W | d(x)<B'}
    return B',U
```

$k = \log^{1/3} n$, $t = \log^{2/3} n$, $m = O(n)$

Time:

$O(nk\cdot(\log n)/t + n\cdot(\log n)/k + m\cdot t) = O(m\cdot\log^{2/3} n)$



THANK YOU!